Smart Marking Setup Guide

From an empty Railway account to students reading their feedback on a phone — every screen is the real app.

VERSION 7 · NOVEMBER 2026 ABOUT 20 MINUTES NO CODING NEEDED

STEP 01 Before you start

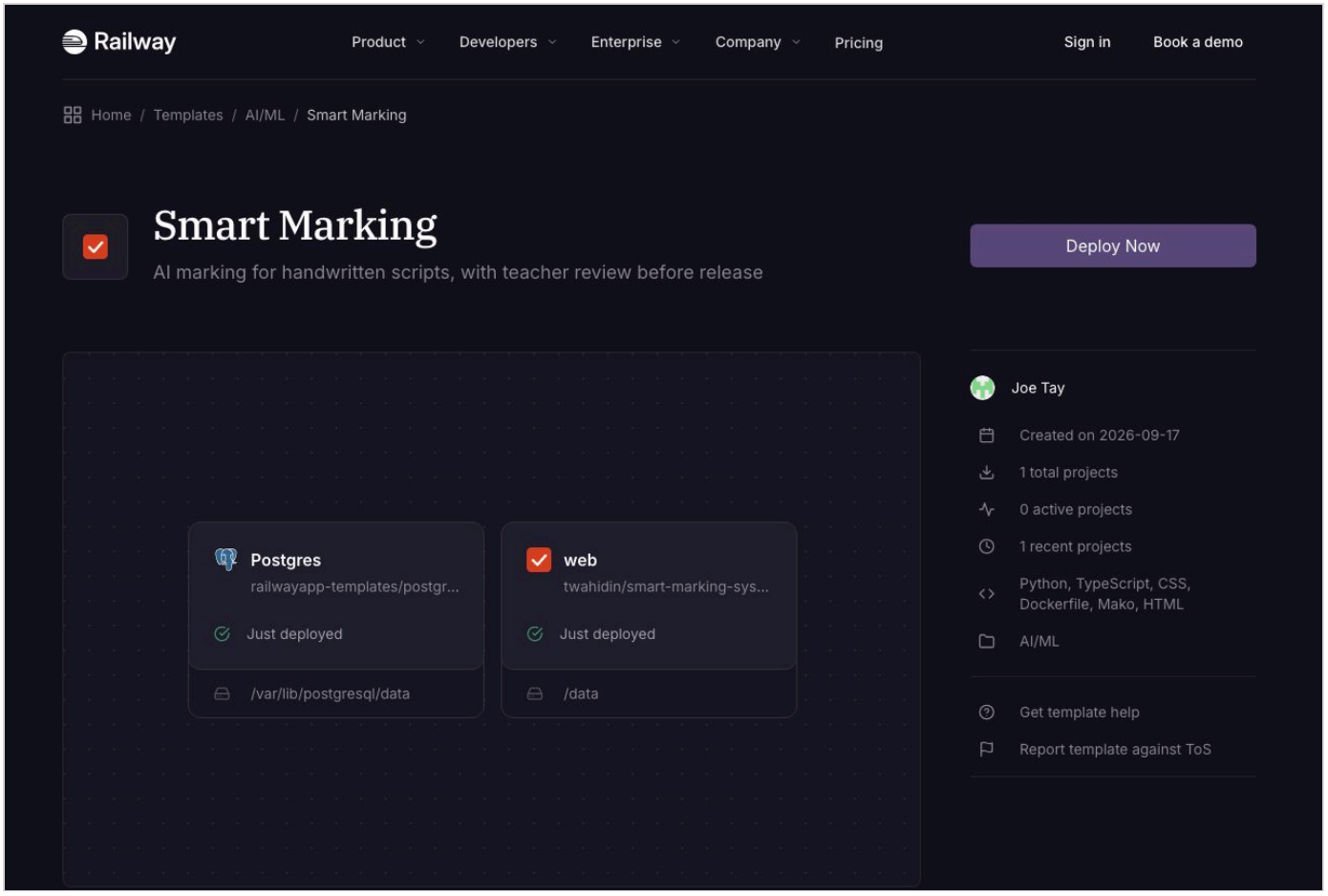
You need three things: a **Railway account** (railway.com — the Hobby plan is enough for a school department), an **API key for one AI provider** that can read images, and about twenty minutes. Free keys that work: Google Gemini's free tier (aistudio.google.com/apikey) and OpenRouter's `:free` models. Paid options: TokenRouter, OpenRouter, OpenAI, Anthropic, Moonshot (Kimi) and Qwen.

Everything — scripts, marks, classes, your key — lives inside your own Railway project. Nothing is sent anywhere except the model calls you configure.

STEP 02 Deploy on Railway

Open railway.com/deploy/smart-marking-1 and click **Deploy Smart Marking**. Railway creates two services — `web` (the app) and `Postgres` (the database) — plus a small disk for page images. The first build takes two to three minutes.

Nothing to copy from the dashboard: there is no password to read and no key to paste in Railway. To open the app, click the `web` service → **Settings** → **Networking** and follow the public domain (it looks like `web-production-xxxx.up.railway.app`). The first person to open it sets everything up on the website — do it straight after deploying.

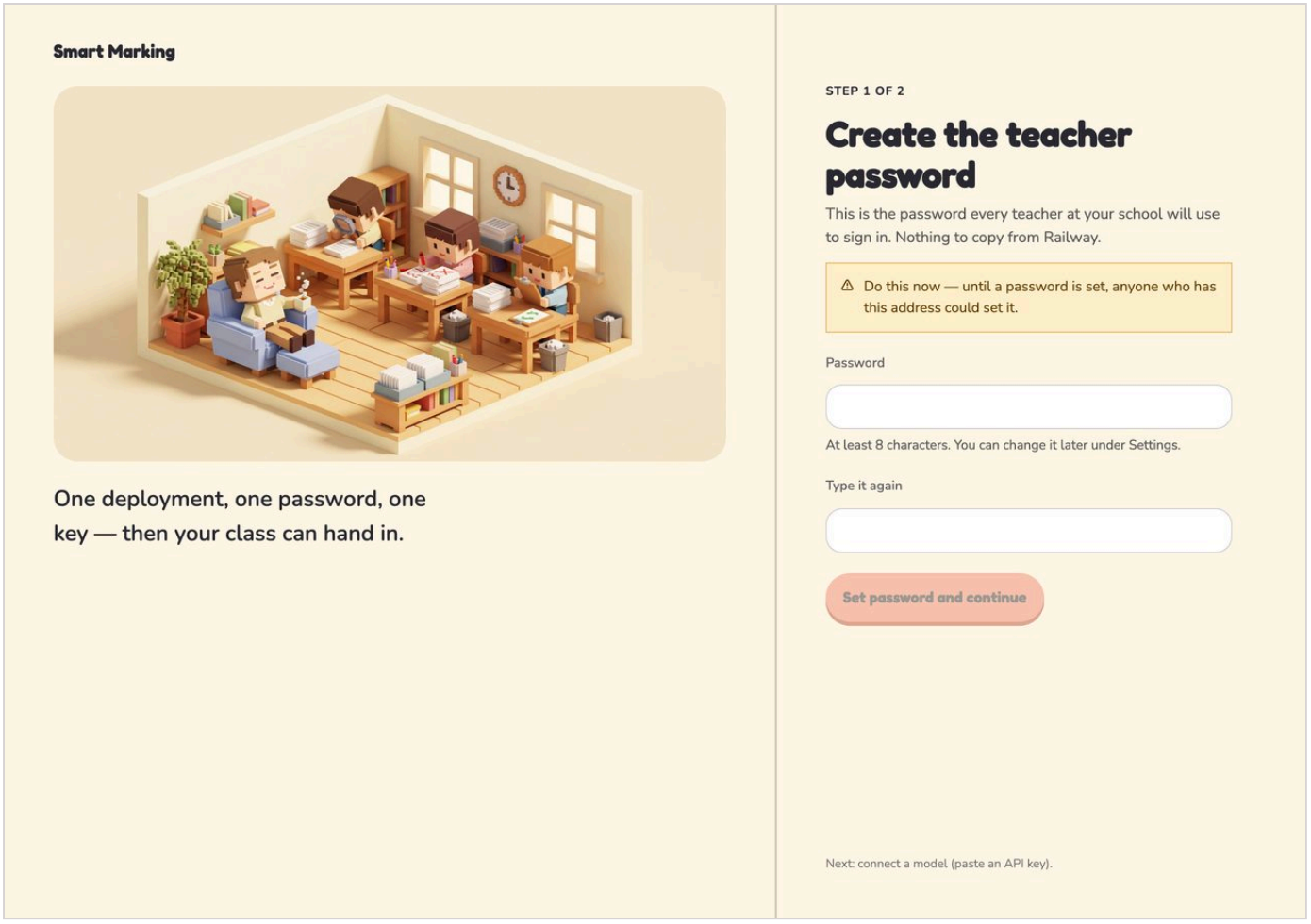


The template page on Railway. One click creates the app, the database and the storage.

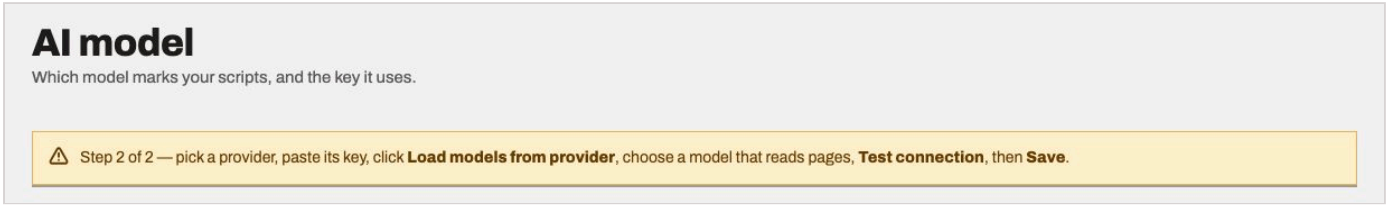
STEP 03 First visit: set the password, then connect a model

Opening a brand-new deployment shows a two-step wizard. **Step 1** asks you to create the teacher password — the one every teacher at your school will use. Do it as soon as the deploy finishes: until a password exists, anyone who has the address could set it.

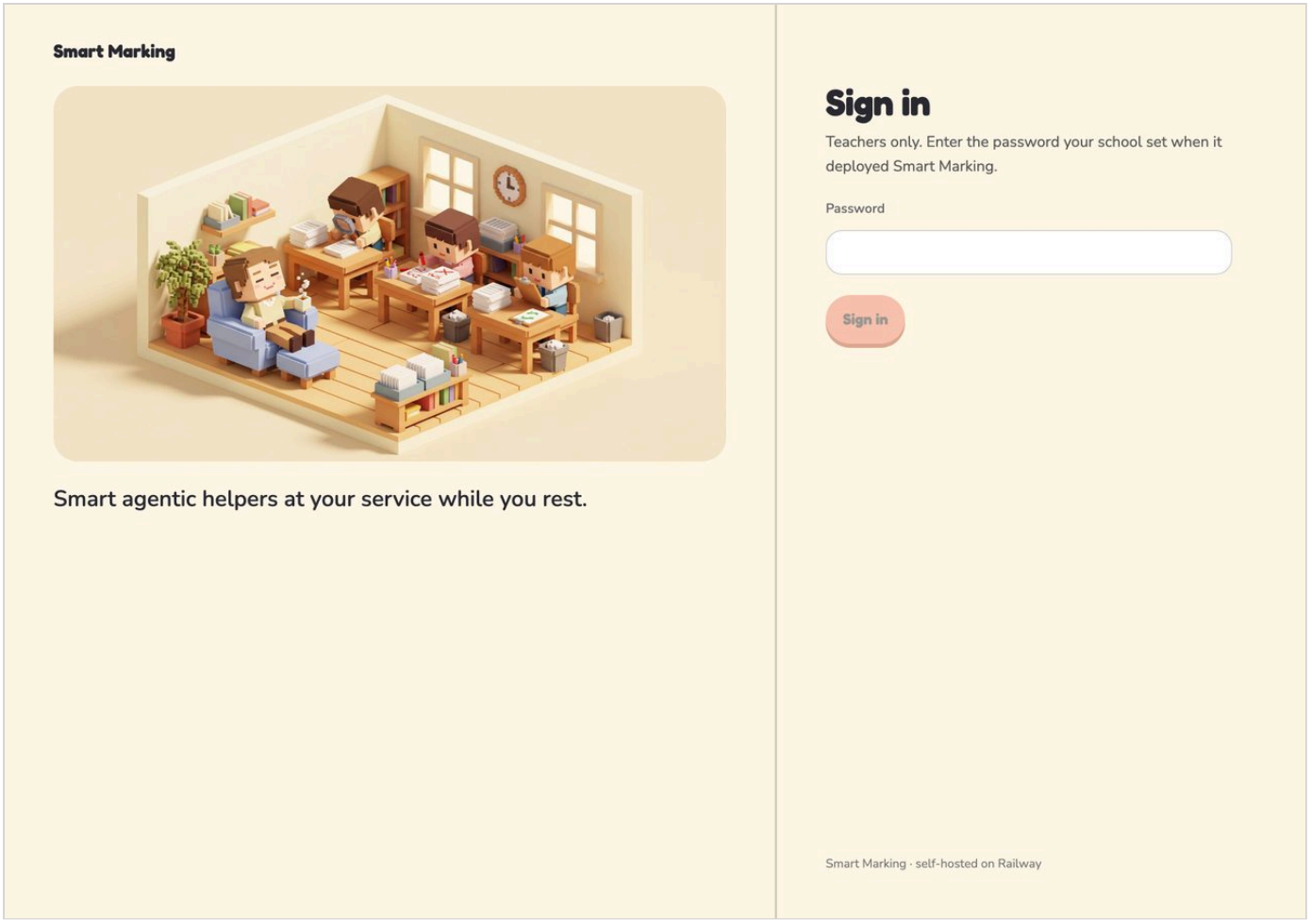
Step 2 is the Settings page: pick a provider, paste its key, load the models, test the connection and save (the next section walks through it). After that, everyone signs in with the password; change it any time under **Settings → Teacher password**. Forgot it? Set a `TEACHER_PASSWORD` variable on the `web` service in Railway — it overrides the website password until you remove it.



Step 1 of the wizard: create the teacher password. No dashboard visit needed.



Step 2 lands on Settings with the checklist for connecting a model.



Every later visit: the sign-in page, with the crew marking while the teacher rests. Teachers only — students never sign in.

STEP 04 Connect a model

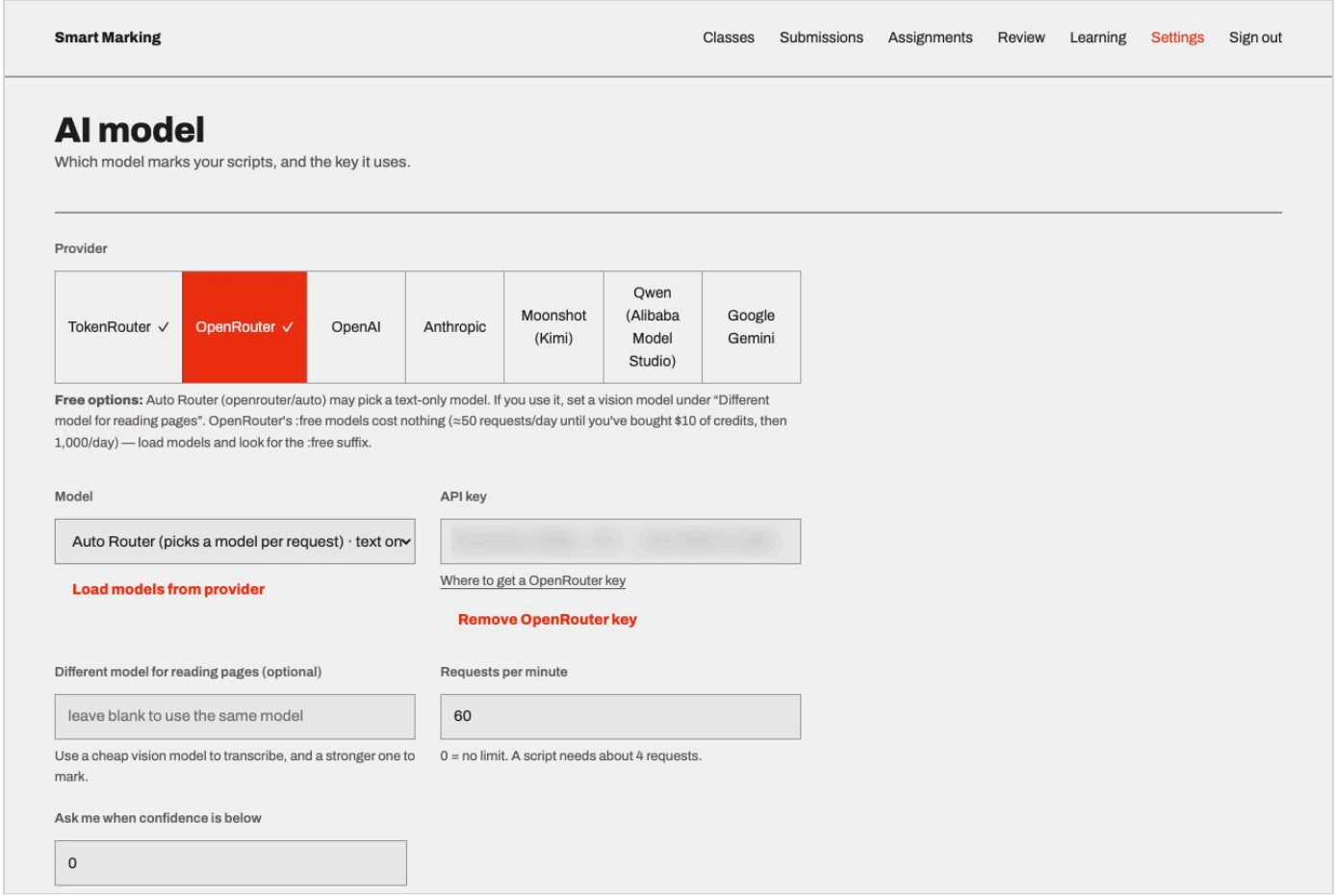
Go to **Settings**. Pick a provider, paste its key, then click **Load models from provider** to see exactly which models your key can use. Choose a model marked *reads pages*, click **Test connection**, and **Save**.

Keys are kept per provider (a ✓ on the tile means a key is saved), so you can switch between providers without re-entering them. **Requests per minute** protects free keys with low rate limits — set it to 10 for Gemini's free tier.

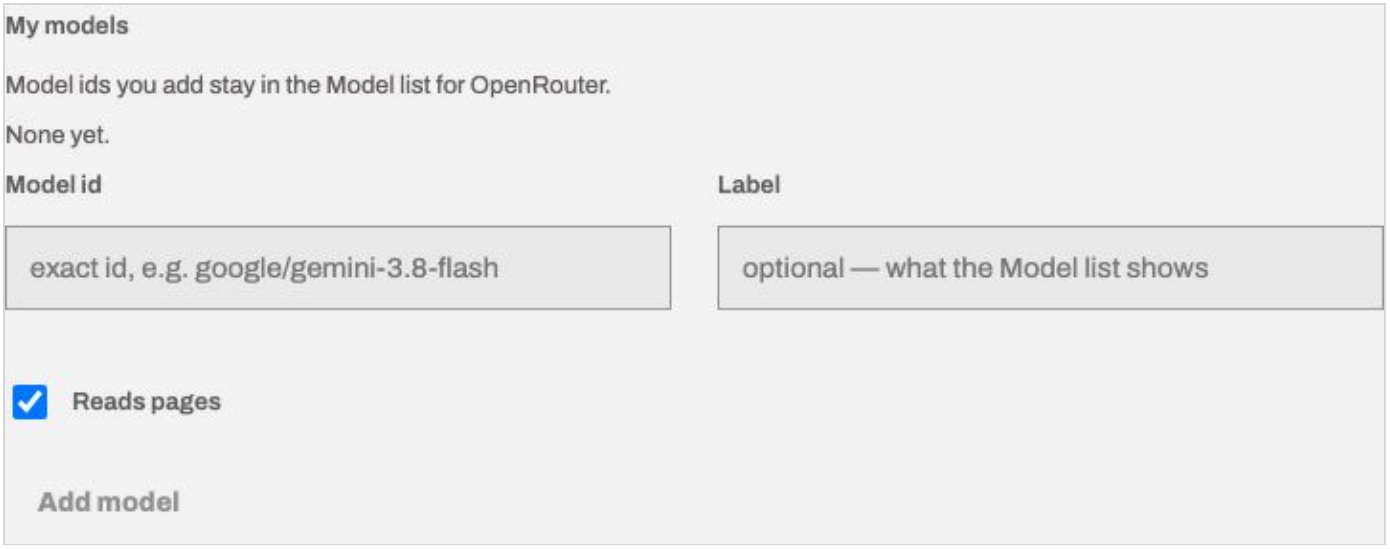
Current models. Each provider's list starts with its current models: Claude Opus 5.5, Sonnet 5.5, Haiku 5.5 and Fable 5.1 under Anthropic; GPT-6 Luna, GPT-6.1 Sol and GPT-6 Astra under OpenAI; Kimi K3 under Moonshot; Gemini 3.8 Flash, 3.5 Flash-Lite and 3.1 Pro under Google. OpenRouter and TokenRouter offer the same mix through one key, cheapest first. OpenRouter also lists **Jev Router**, which picks a model for each request, so its cost varies from script to script. If the image check in

Test connection fails once, run it again: good models occasionally misread its test number.

My models. OpenRouter and TokenRouter host hundreds of models beyond the curated list. Under those two providers a *My models* list lets you add any model id (with a label and whether it reads pages); it then appears in every model picker, here and in the assignment editor. Each id returned by *Load models from provider* gets a **+ Add** button.



Settings: provider, model, key. Test connection makes one small text call and one image call.



My models: your own OpenRouter or TokenRouter model ids, available everywhere a model is picked.

STEP 05 Create an assignment

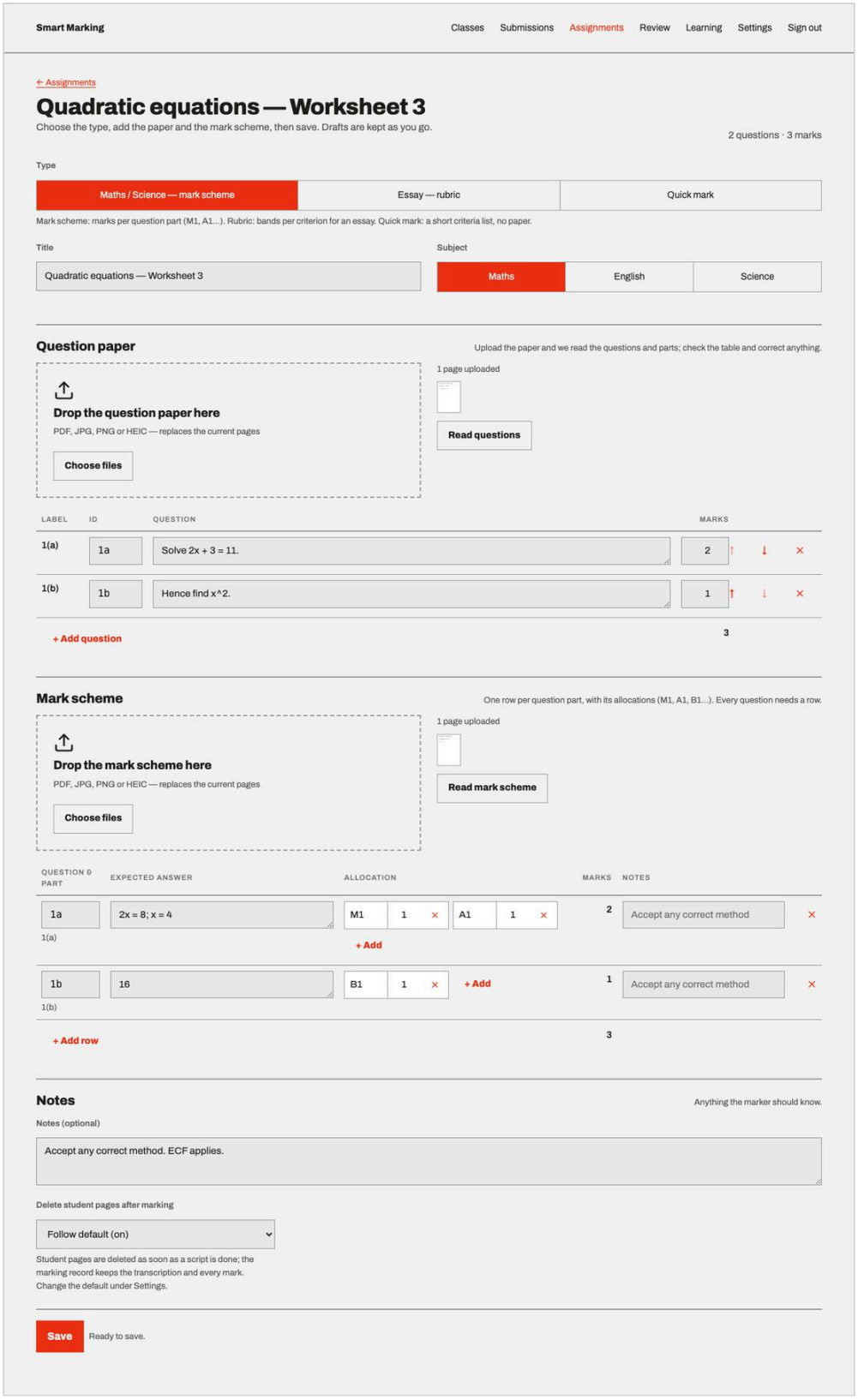
Go to **Assignments → + New assignment**. Choose the type: *Maths / Science — mark scheme* for structured answers, *Essay — rubric* for open responses, or *Quick mark* for a short criteria list with no paper.

Give it a title, drop the **question paper** (PDF or photos) and click **Read questions** — the app transcribes every question and part into a table you can correct. Then drop the **mark scheme** and click **Read mark scheme**: each row shows the expected answer and its mark allocation (M1, A1, B1…). Add any notes the marker should know ("ECF applies", "accept any correct method"), check the tables, and **Save**.

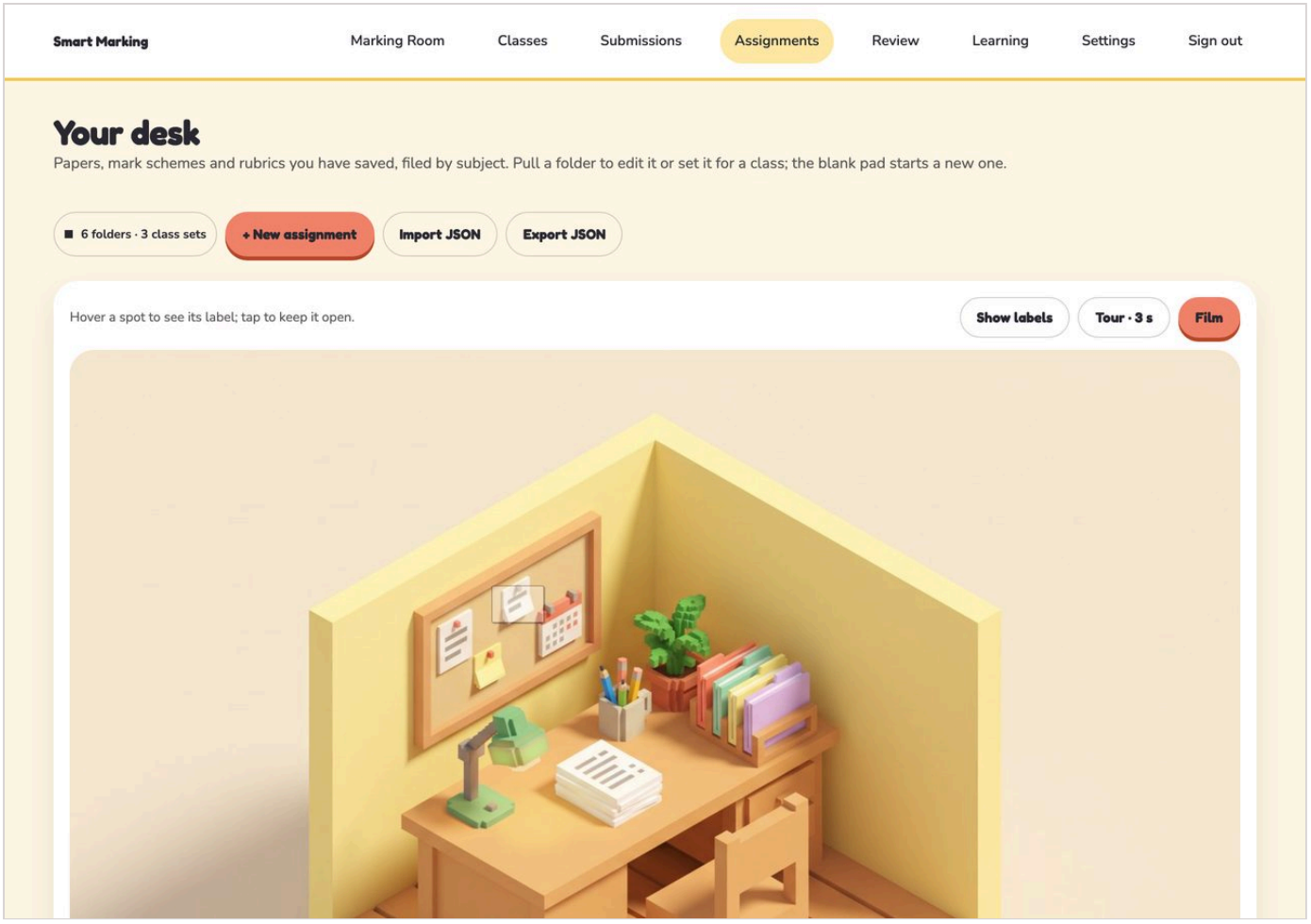
Saved assignments go into a bank you can reuse across classes and years.

Assignments opens on your desk: the folders in the rack are your saved assignments, filed by subject; the blank pad starts a new one; the pinboard opens whatever is due this week; and the lamp lights up while scripts are being marked. Hover or tap a spot to see its label, or turn on **Show labels**.

Subjects. Maths, English and Science as before, plus **MT** (Mother Tongue — pick Chinese, Malay or Tamil; the script is read in that language and the student's feedback is written in it, while everything you see stays in English) and **Computing**, whose students can hand in program files as well as photos.



The assignment editor after Read questions and Read mark scheme. Every cell is editable.



Assignments is your desk. Every spot is a control: a subject's folders, the blank pad, the pinboard, the lamp.



A Computing assignment: students may hand in .py, .sb3 and .xlsx files.



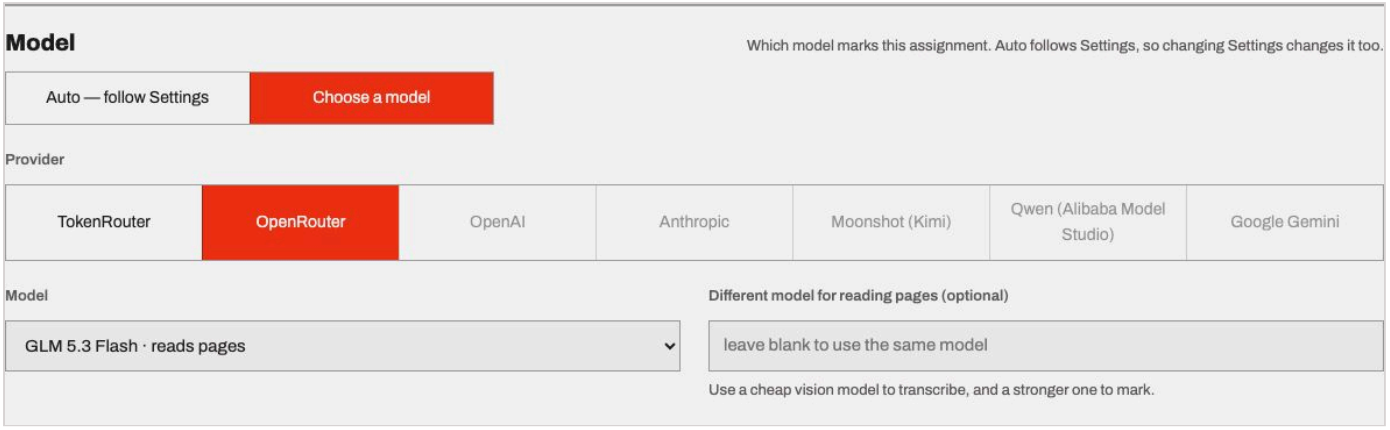
An MT assignment names its language.

STEP 06 Choose the model per assignment

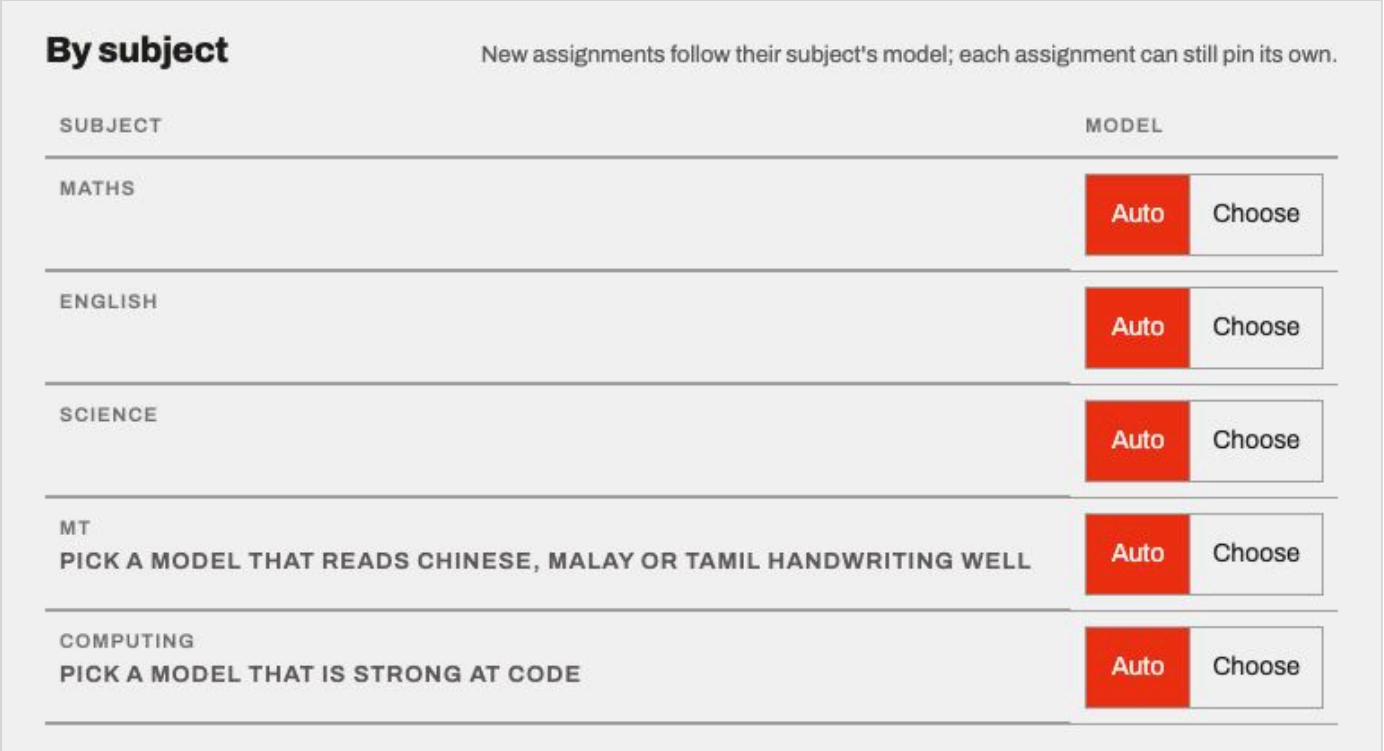
Every assignment follows Settings by default (*Auto — follow Settings*): change the model in Settings and every assignment changes with it. For one assignment that deserves something different — a hard paper for a stronger model, a quick quiz for a cheap one — open it, go to **Model** and pick *Choose a model*. Only providers with a saved key can be selected; choose the model, optionally a different one for reading pages, and **Save**. The assignment bank shows the choice as a small caption.

Removing a provider's key while assignments still use it asks you to confirm first — those assignments would fail to mark until you point them at another model.

A default per subject. Under **Settings → By subject** you can give each subject its own default — a model that reads Chinese, Malay or Tamil handwriting well for MT, one that is strong at code for Computing — and new assignments in that subject follow it. The order is always: the assignment's own choice, then its subject's default, then Settings.



The Model section of an assignment: Auto, or a provider and model of your own.



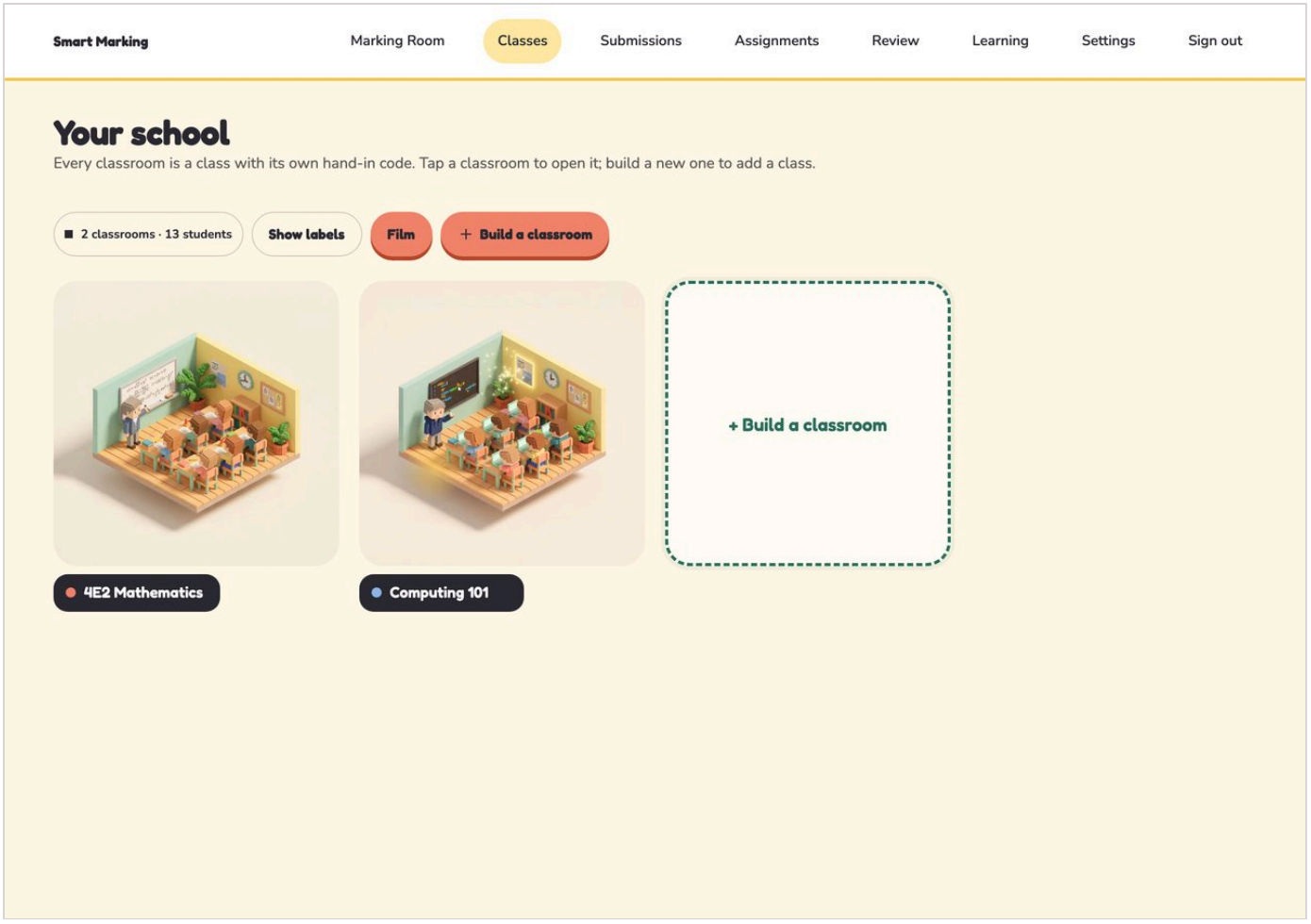
Settings → By subject: one default model per subject, Auto by default.

STEP 07 Create a class and load the classlist

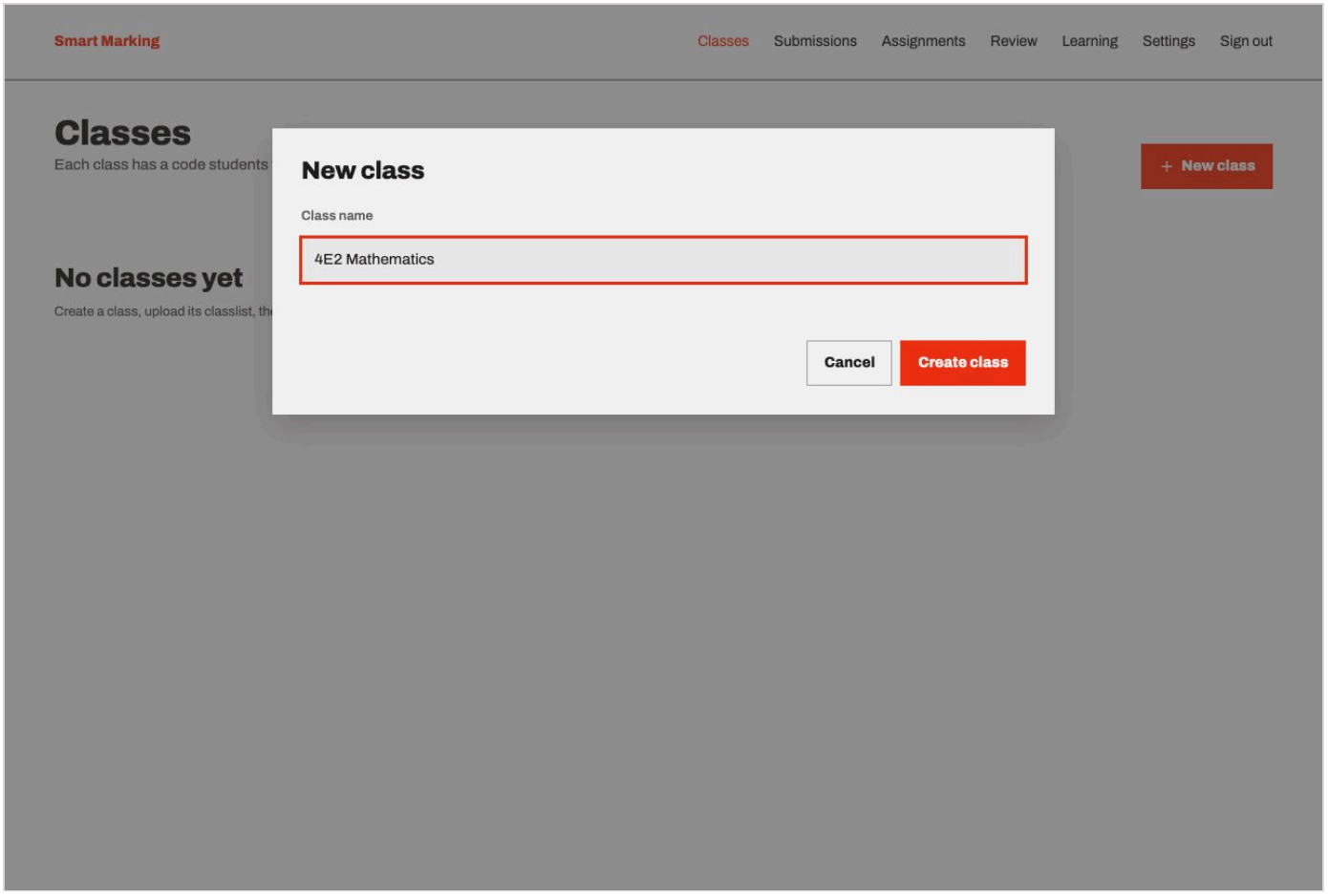
Classes is your school: one classroom per class, painted for its subject, and an empty plot to build the next one. Go to **Classes → Build a classroom** and name it the way your school does ("4E2 Mathematics"). Pick a **Subject**: it chooses the painting on the classroom's tile, and you can change it later under the class's **Settings** tab. Every class gets a 4-character code — this one is `4KEF` — that students will type.

On the **Students** tab, upload a CSV with two columns, `name` and `reg_no` (register number). The preview flags duplicate or missing numbers before anything is saved; click **Confirm classlist** when it looks right.

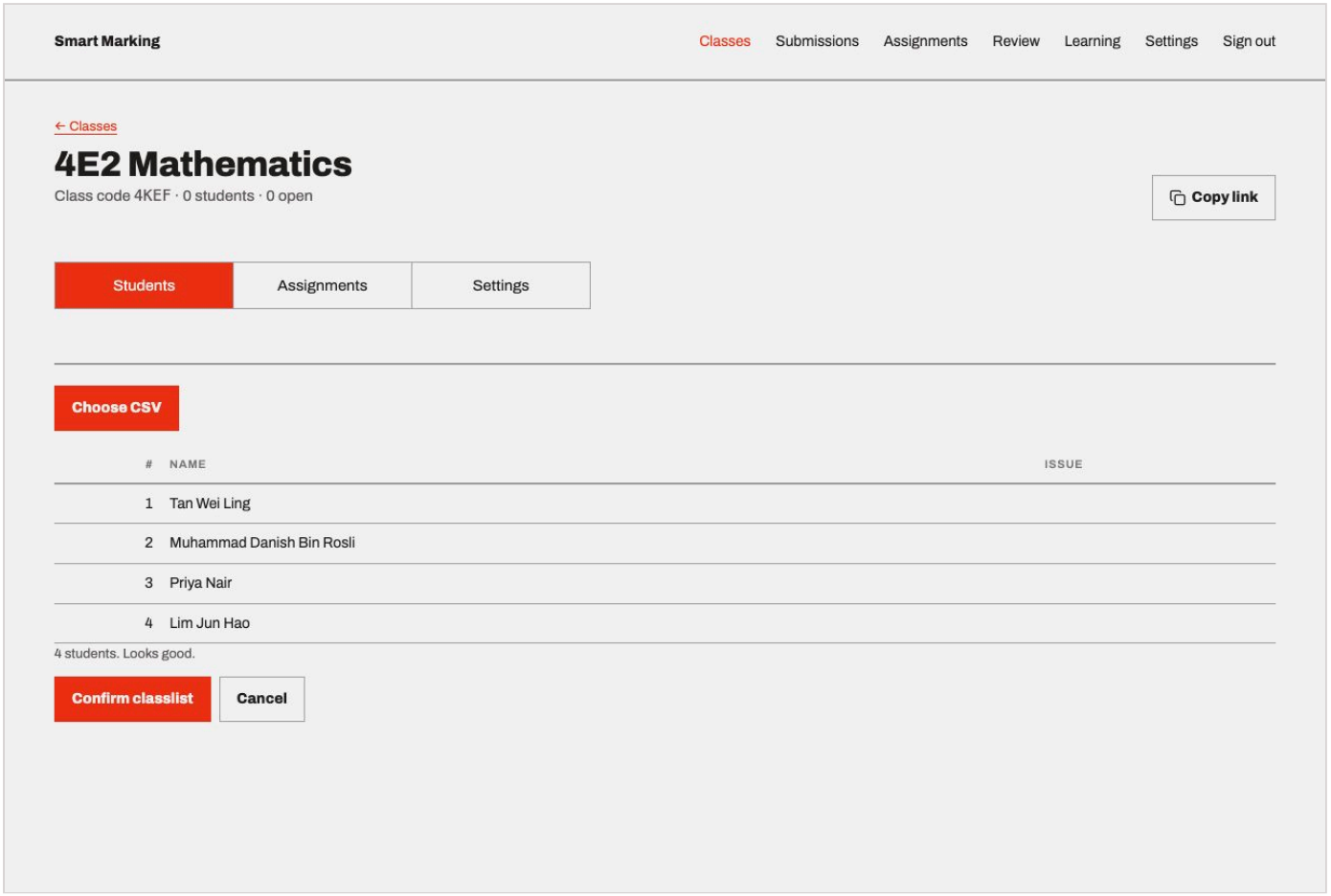
Then click **Copy link** and paste it into Google Classroom (or wherever your students look). The link opens the app with the class code already filled in.



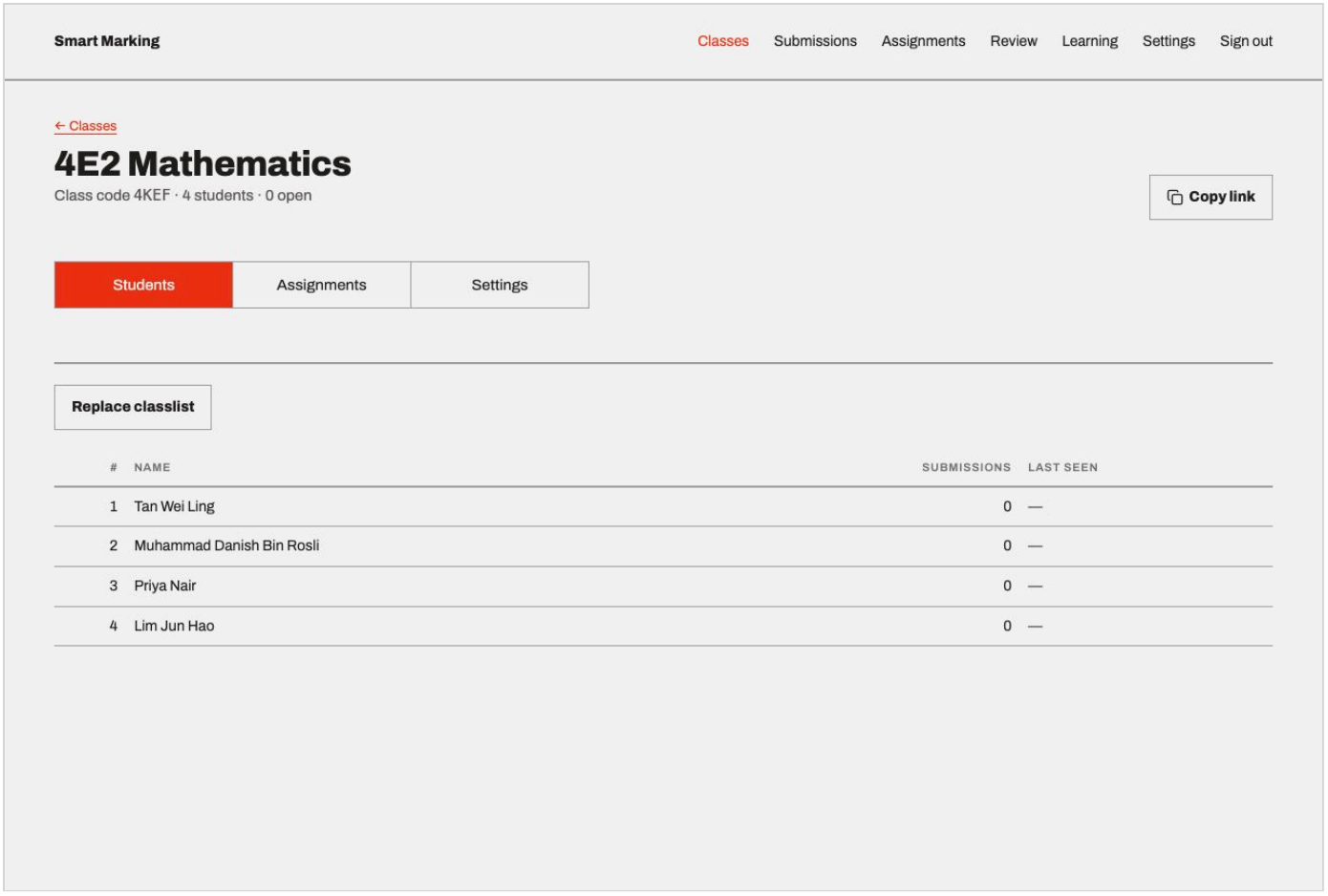
Classes is your school: a classroom per class, and an empty plot for the next one.



Creating a class.



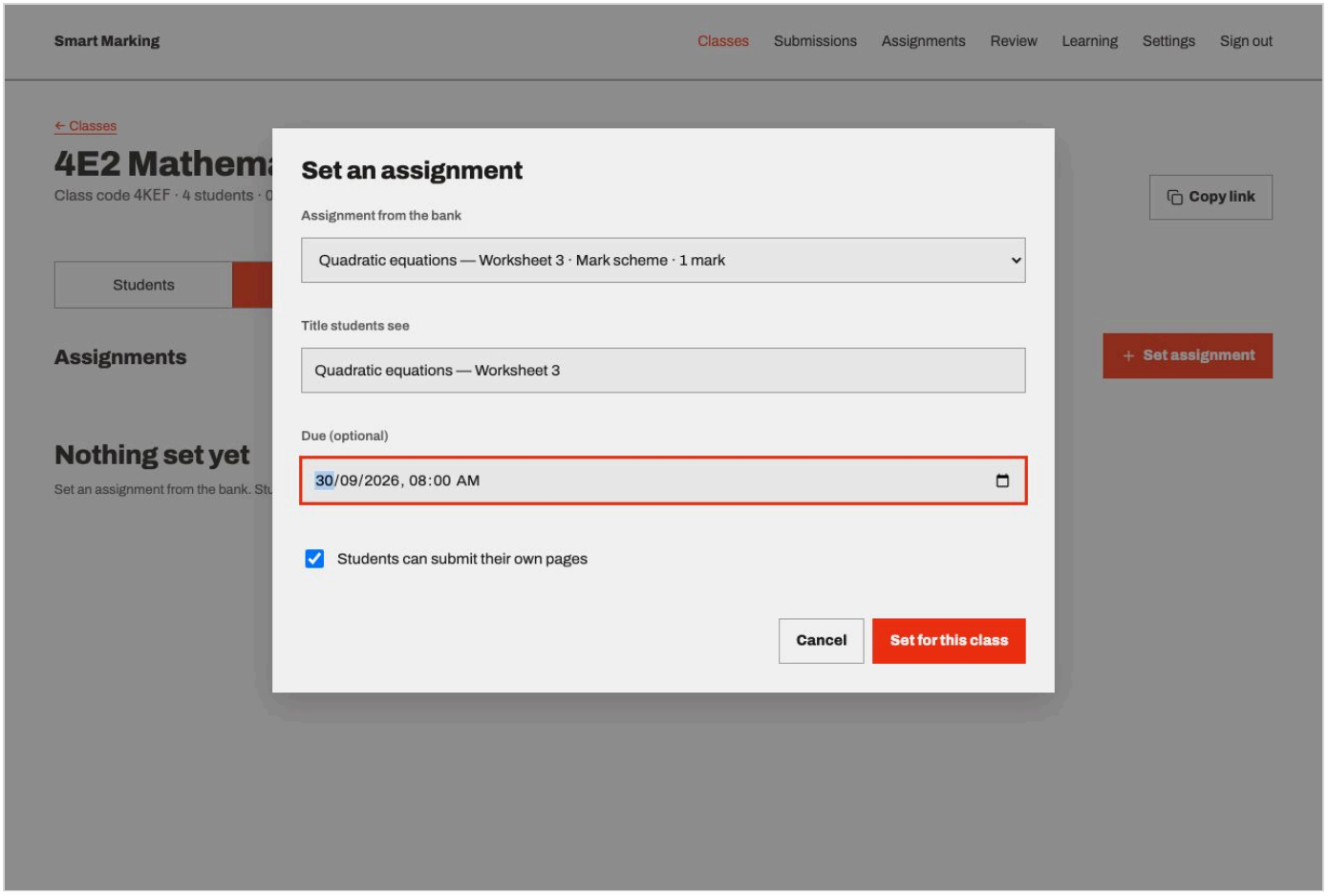
CSV preview — issues are shown per row before you confirm.



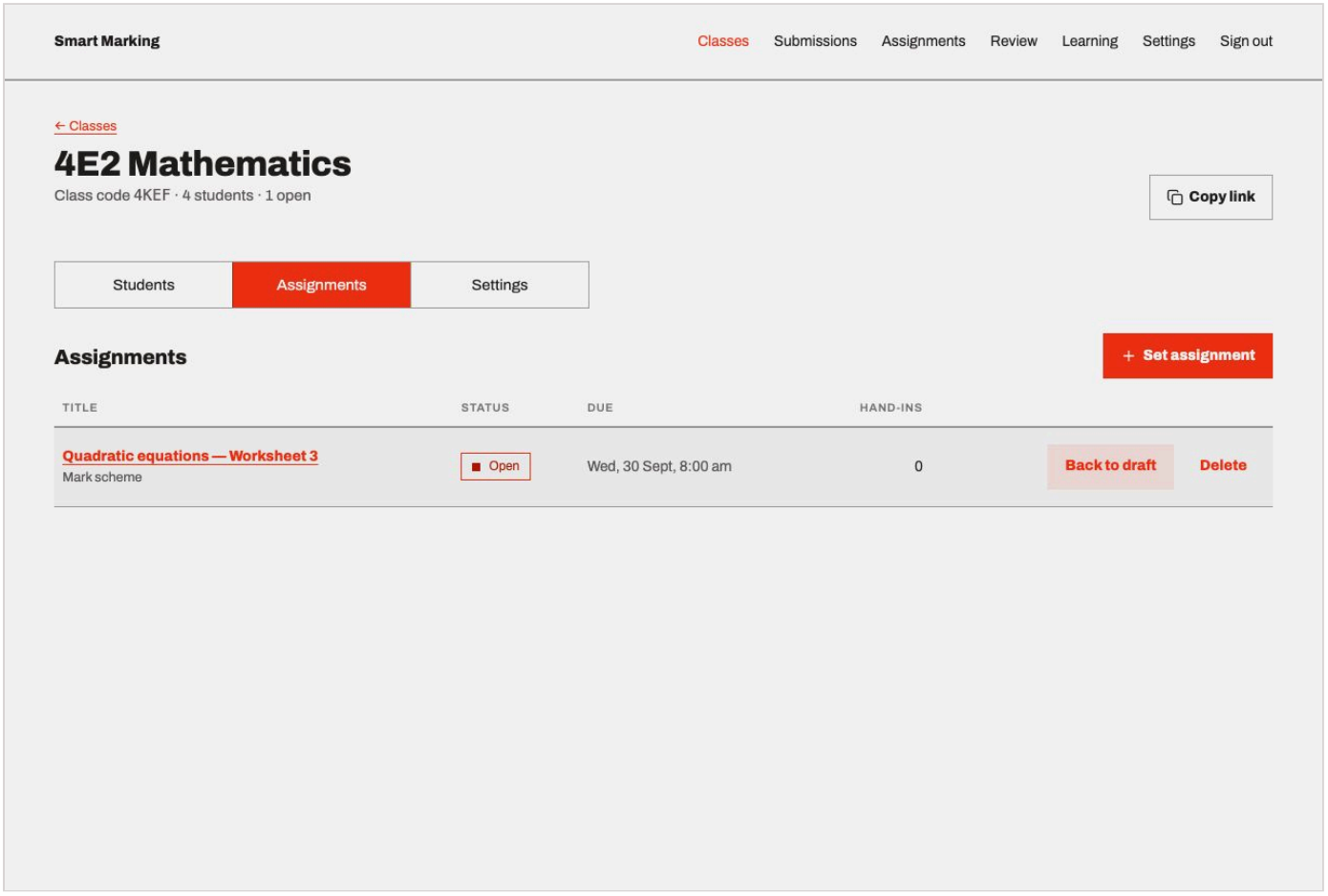
The classlist, the class code and the Copy link button.

STEP 08 Set the assignment for the class

On the class's **Assignments** tab click **Set assignment**, pick one from the bank, add a due date and click **Set for this class**. It starts as a *Draft*, invisible to students. Click **Open** when you want students to hand in.



Setting an assignment from the bank.

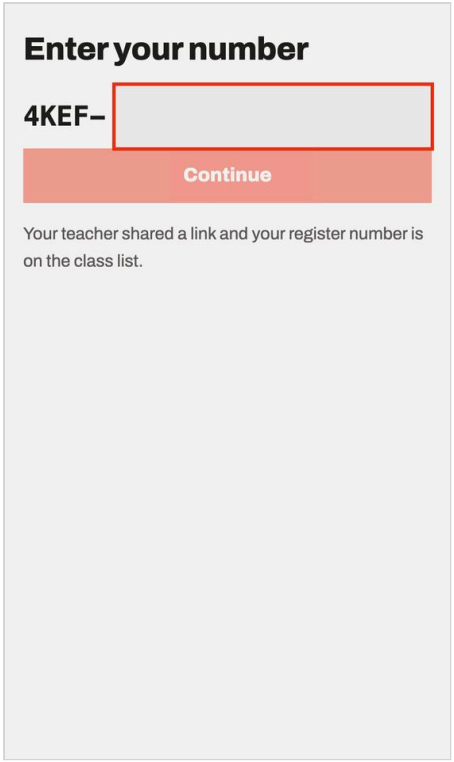


Opened — students can now see it.

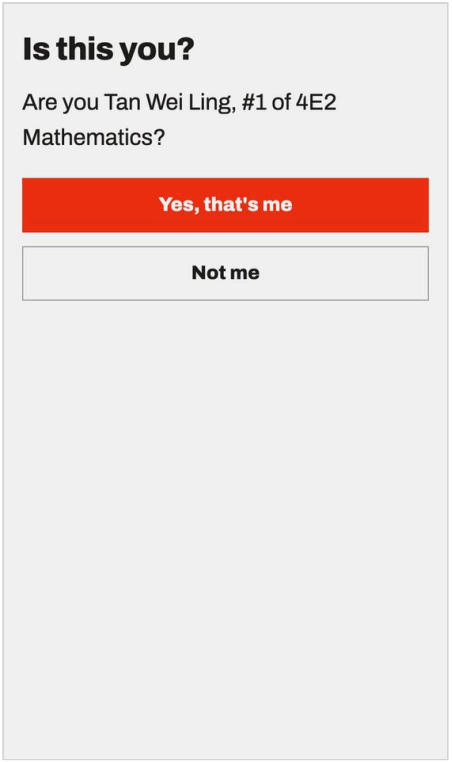
STEP 09 Students hand in from their phones

A student opens the class link, types their register number and confirms their name. No account, no password: anyone with the class code and a number can see that student's work — the same trade-off as a paper script left on a desk.

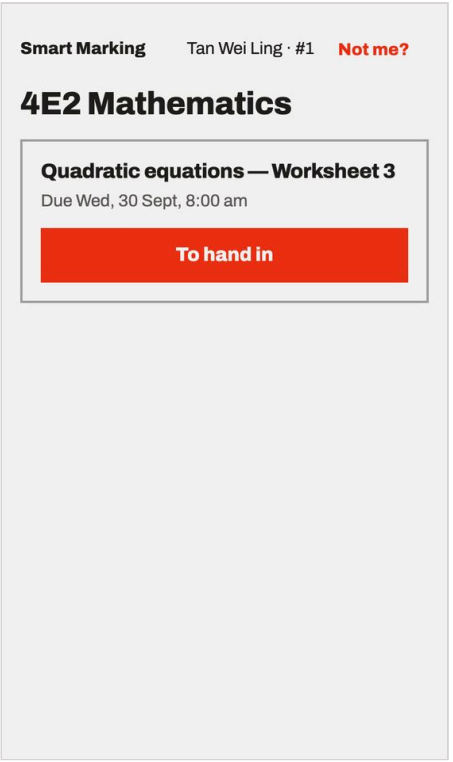
Hand in takes photos with the camera or from the gallery. Pages can be re-ordered or deleted before handing in; photos are shrunk on the phone before upload, so it works on a weak signal. Up to 20 pages per script. Once handed in, the student sees a calm "Handed in" note — marking starts straight away in the background.



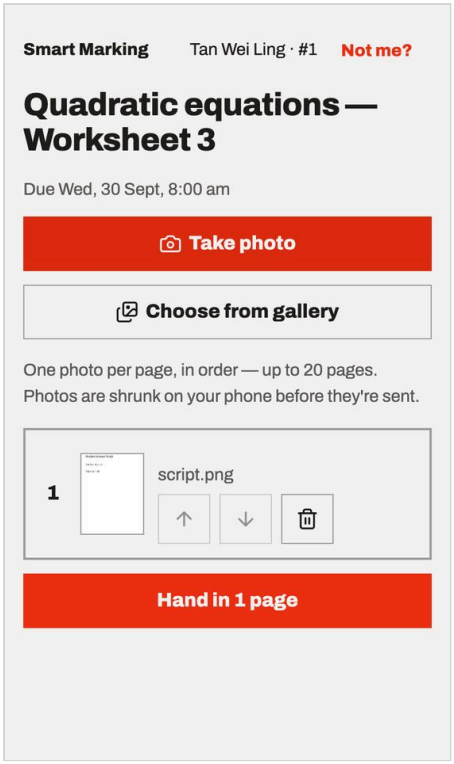
Enter your number.



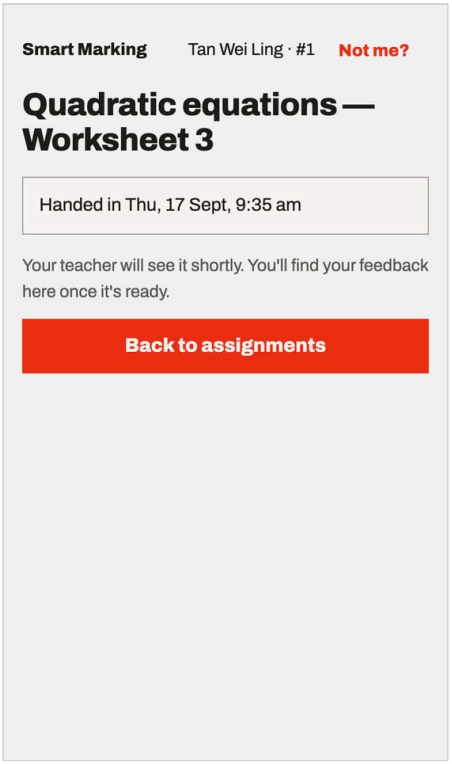
Confirm it's you.



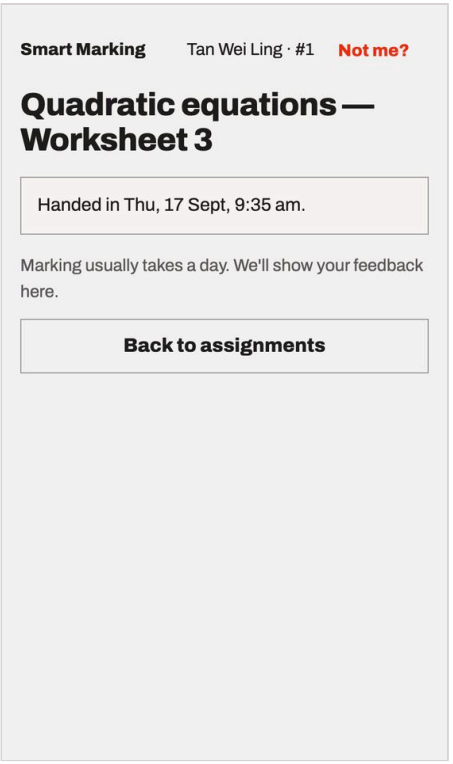
Home: what to hand in.



Pages in order, ready to hand in.



Done.

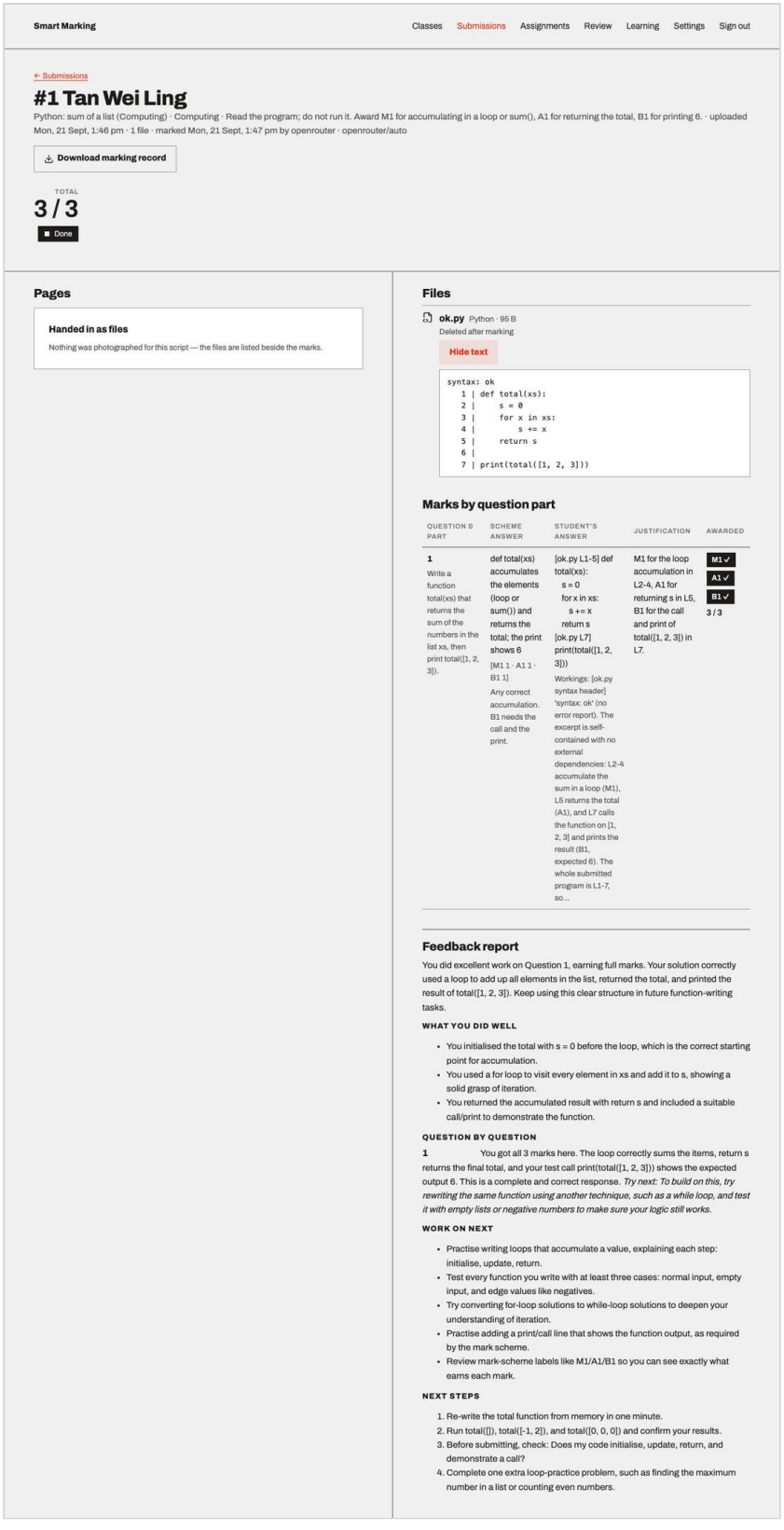


Waiting for the teacher.

STEP 10 Computing: hand in files, not just photos

For a Computing assignment the hand-in page has a third button, **Add files**: Python (`.py`), Scratch (`.sb3`) and Excel (`.xlsx`) files, or a `.zip` of them, alone or together with photos of anything written on paper — up to 20 items and 12 files per hand-in, 2 MB per file. Teachers can drop the same files for a student from the class page.

The app *reads* the work; it never runs it. Python is shown with line numbers and a syntax check, Scratch projects as their blocks in words (*when green flag clicked · repeat (10) · move (10) steps*), and spreadsheets cell by cell with the formula and its value. The marker cites the file and line for every mark, and the submission page keeps that text even after the files are deleted.



Smart Marking

Classes Submissions Assignments Review Learning Settings Sign out

← Submissions

#1 Tan Wei Ling

Python: sum of a list (Computing) · Computing · Read the program; do not run it. Award M1 for accumulating in a loop or sum(), A1 for returning the total, B1 for printing 6. · uploaded Mon, 21 Sept, 1:46 pm · 1 file · marked Mon, 21 Sept, 1:47 pm by openrouter · openrouter/auto

Download marking record

TOTAL
3 / 3
Done

Pages

Handed in as files

Nothing was photographed for this script — the files are listed beside the marks.

Files

ok.py Python · 95 B
Deleted after marking

Hide text

```
syntax: ok
   1 | def total(xs):
   2 |     s = 0
   3 |     for x in xs:
   4 |         s += x
   5 |     return s
   6 |
   7 | print(total([1, 2, 3]))
```

Marks by question part

QUESTION & PART	SCHEME ANSWER	STUDENT'S ANSWER	JUSTIFICATION	AWARDED
1 Write a function total(xs) that returns the sum of the numbers in the list xs, then print total([1, 2, 3]).	def total(xs) accumulates the elements (loop or sum()) and returns the total; the print shows 6 [M1 1 · A1 1 · B1 1] Any correct accumulation. B1 needs the call and the print.	[ok.py L1-5] def total(xs): s = 0 for x in xs: s += x return s [ok.py L7] print(total([1, 2, 3])) Workings: [ok.py syntax header] 'syntax: ok' (no error report). The excerpt is self-contained with no external dependencies: L2-4 accumulate the sum in a loop (M1), L5 returns the total (A1), and L7 calls the function on [1, 2, 3] and prints the result (B1, expected 6). The whole submitted program is L1-7, so…	M1 for the loop accumulation in L2-4, A1 for returning s in L5, B1 for the call and print of total([1, 2, 3]) in L7.	M1 ✓ A1 ✓ B1 ✓ 3 / 3

Feedback report

You did excellent work on Question 1, earning full marks. Your solution correctly used a loop to add up all elements in the list, returned the total, and printed the result of total([1, 2, 3]). Keep using this clear structure in future function-writing tasks.

WHAT YOU DID WELL

- You initialised the total with s = 0 before the loop, which is the correct starting point for accumulation.
- You used a for loop to visit every element in xs and add it to s, showing a solid grasp of iteration.
- You returned the accumulated result with return s and included a suitable call/print to demonstrate the function.

QUESTION BY QUESTION

1 You got all 3 marks here. The loop correctly sums the items, return s returns the final total, and your test call print(total([1, 2, 3])) shows the expected output 6. This is a complete and correct response. *Try next: To build on this, try rewriting the same function using another technique, such as a while loop, and test it with empty lists or negative numbers to make sure your logic still works.*

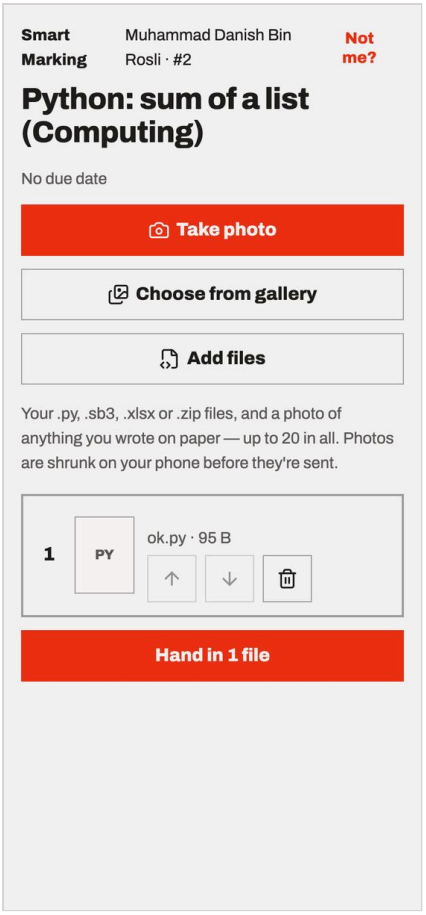
WORK ON NEXT

- Practise writing loops that accumulate a value, explaining each step: initialise, update, return.
- Test every function you write with at least three cases: normal input, empty input, and edge values like negatives.
- Try converting for-loop solutions to while-loop solutions to deepen your understanding of iteration.
- Practise adding a print/call line that shows the function output, as required by the mark scheme.
- Review mark-scheme labels like M1/A1/B1 so you can see exactly what earns each mark.

NEXT STEPS

1. Re-write the total function from memory in one minute.
2. Run total([]), total([-1, 2]), and total([0, 0, 0]) and confirm your results.
3. Before submitting, check: Does my code initialise, update, return, and demonstrate a call?
4. Complete one extra loop-practice problem, such as finding the maximum number in a list or counting even numbers.

A marked Python file: the rendered text, the marks with their citations, and the feedback.

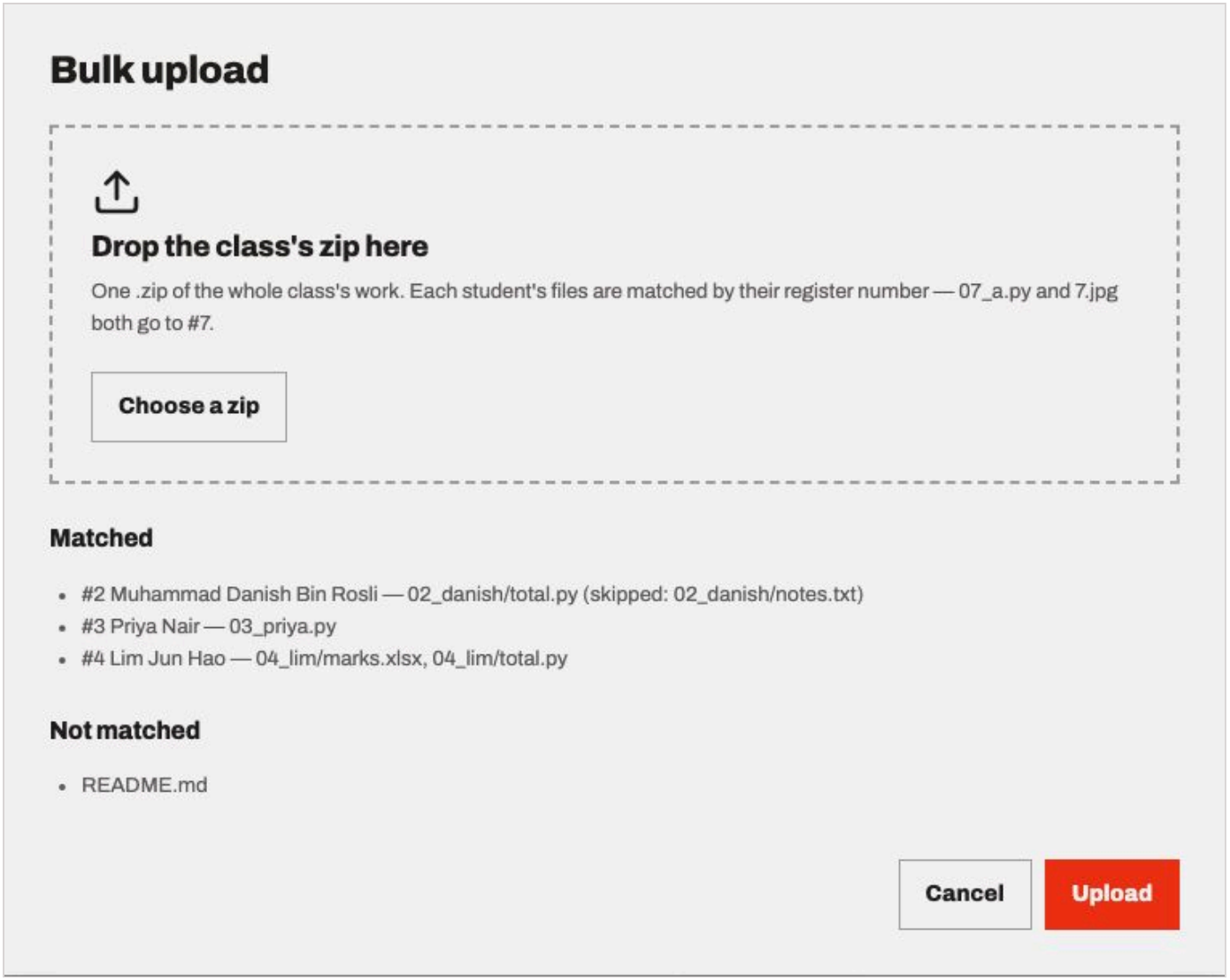


Add files sits next to the camera buttons.

STEP 11 Bulk upload a whole class

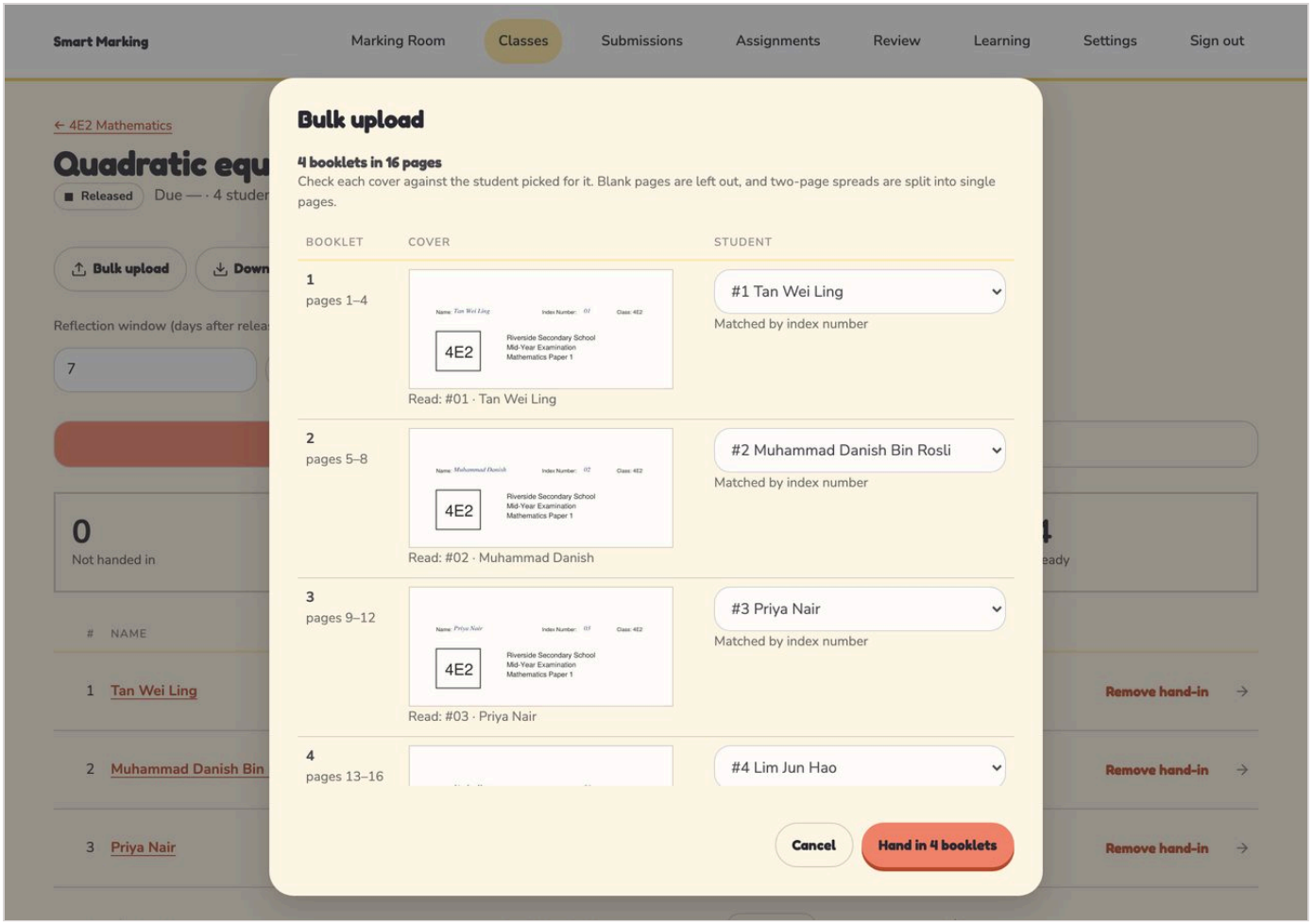
Instead of uploading student by student, put the whole class's work in one `.zip` where each file or folder starts with the register number — `07_amirah.py`, `07/…`, `7 - amirah/` — and click **Bulk upload** on the class assignment page. The preview shows who was matched, which files were skipped and what could not be matched, before anything is saved.

Students who have already handed in are skipped unless you tick **Replace existing hand-ins**. Photos in a bulk zip are not shrunk in the browser, so keep the zip under 50 MB unpacked or split the class in two.



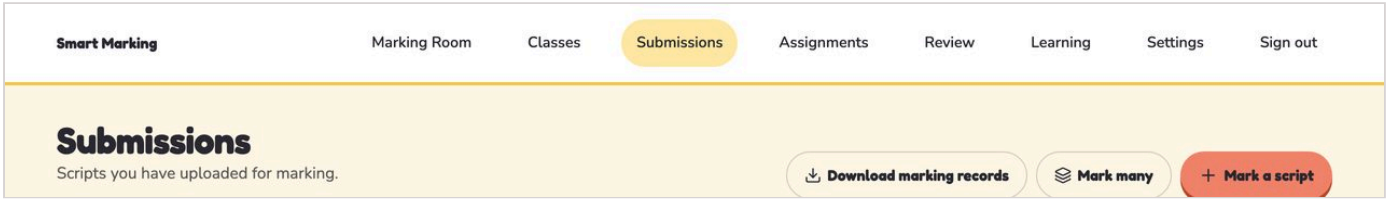
The bulk-upload preview: matched students, skipped files, unmatched entries.

One PDF of the whole class. If your scanner gives you a single PDF of every booklet, drop that instead. The app splits it at each booklet's cover, so booklets can be different lengths, reads the index number and name written there, and proposes the matching student. Check each cover against the student picked for it, fix any that are wrong or unreadable with the dropdown, then click **Hand in**. Two-page spreads are split into single pages, which doubles the detail the marker reads, and blank pages are left out. Scripts already marked in red ink show those marks to the model, so scan clean scripts for an independent mark.

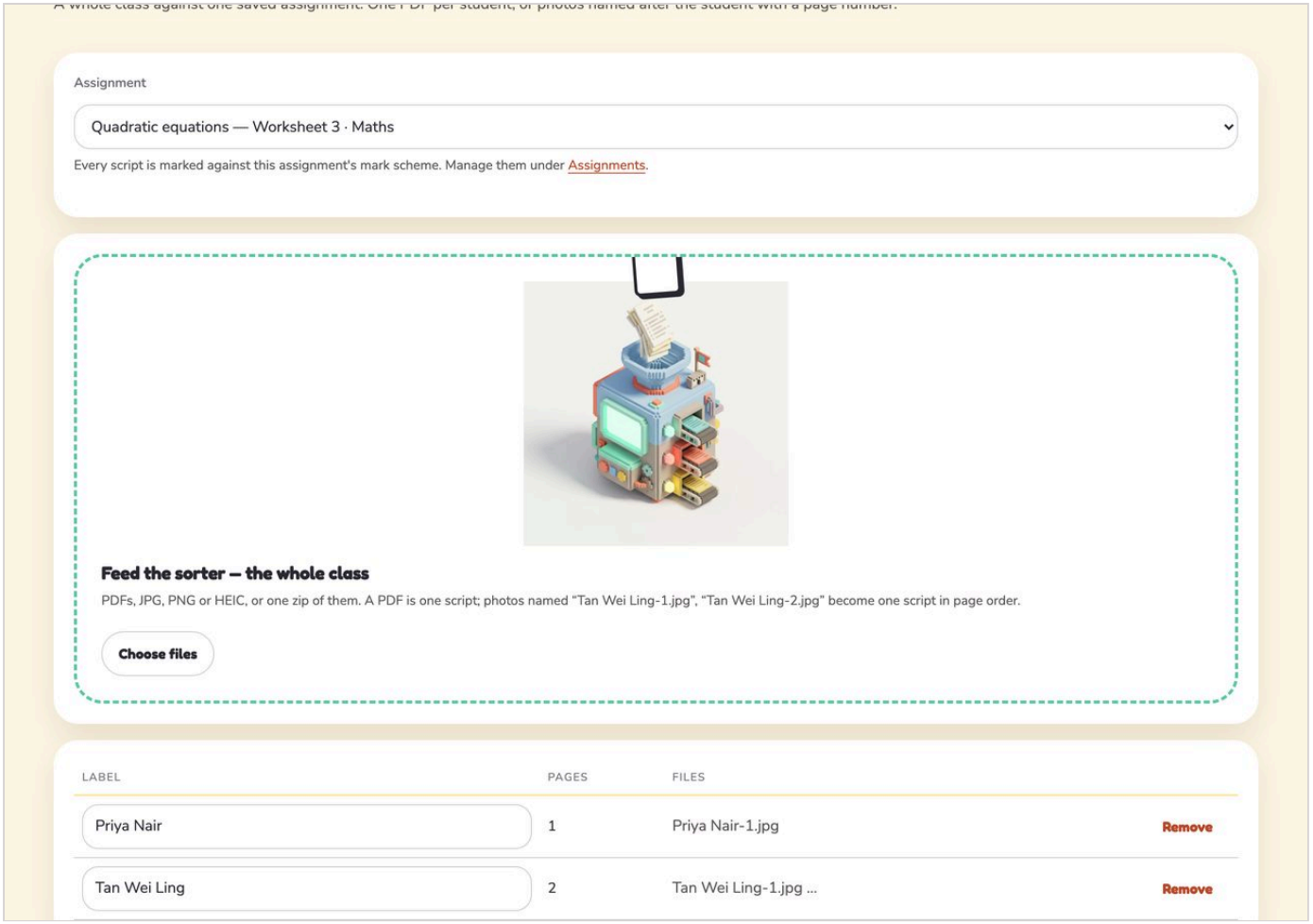


One combined PDF: each booklet's cover, what was read from it, and the student it matched.

Not a class set? Submissions → Mark many marks a pile of scripts against one saved assignment without a class. Drop PDFs, photos or one zip of them: each PDF is one script, and photos named after the student with a page number, such as `Tan Wei Ling-1.jpg` and `Tan Wei Ling-2.jpg`, become one script in page order. Labels come from the file names and can be edited before you start; scripts that fail to upload stay listed with a retry.



Submissions: Mark many sits beside Mark a script.



Mark many: one row per script, with its label, pages and files.

STEP 12 Marking, and the parts that need you

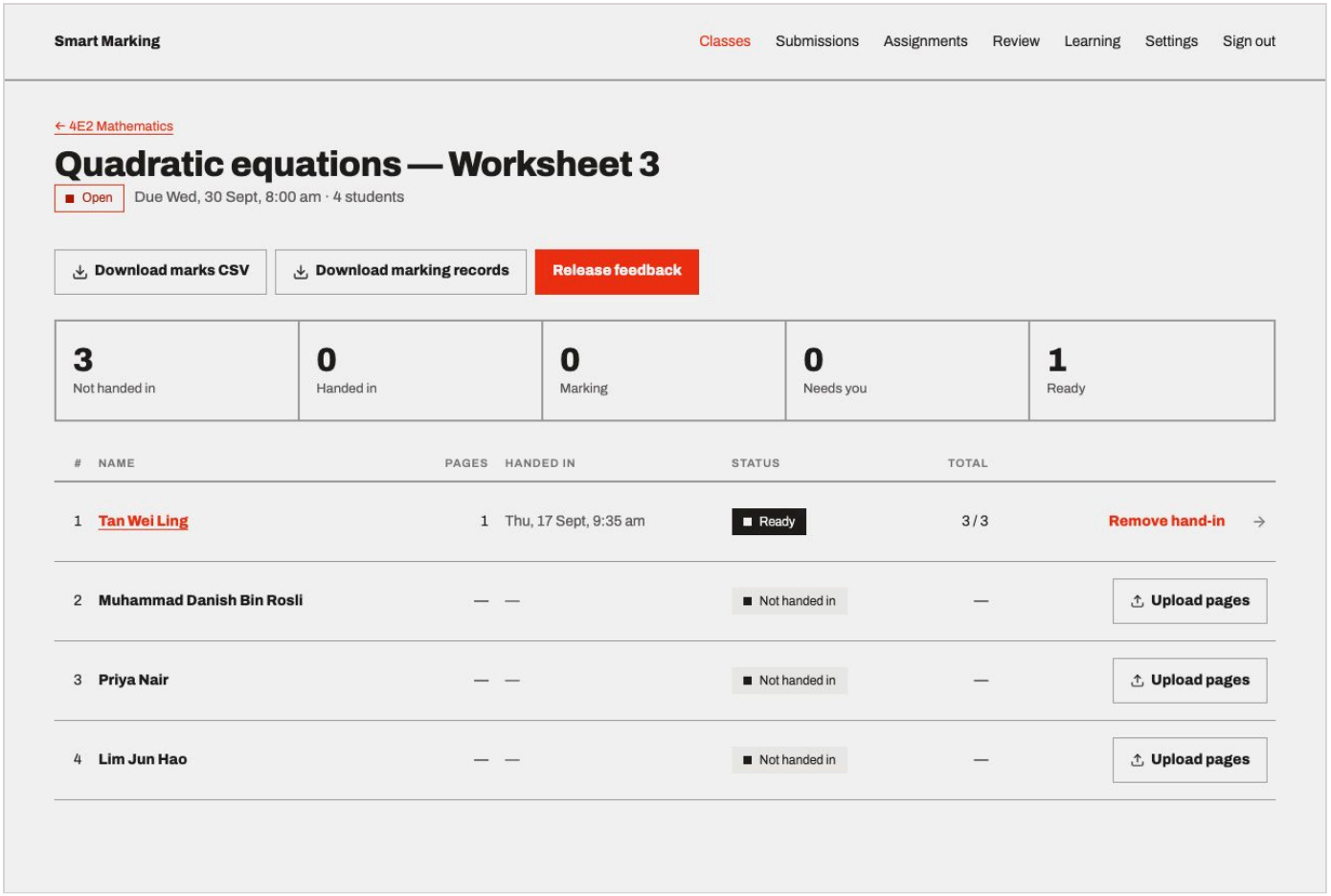
The assignment page shows the roster: who has handed in, what is being marked, and what is ready. You can also **Upload pages** for a student yourself (scanned booklets) and **Remove hand-in** so a student can redo it.

Each script is read by one model, marked against the scheme by a second pass, and checked by an independent reviewer. Anything illegible, not covered by the scheme, or where marker and reviewer disagree goes to **Review** with the reason. There each allocation (M1, A1, B1…) has a mark box that starts at the marker's proposal: change only what you disagree with, or pick the band for an essay, and give a one-line reason — that reason feeds the nightly learning step, so the marker gets closer to your judgement over time. **Save & next** accepts the proposal as it stands. A scheme that gives the same label twice, such as two B1 marks, shows them as B1 (1 of 2) and B1 (2 of 2).

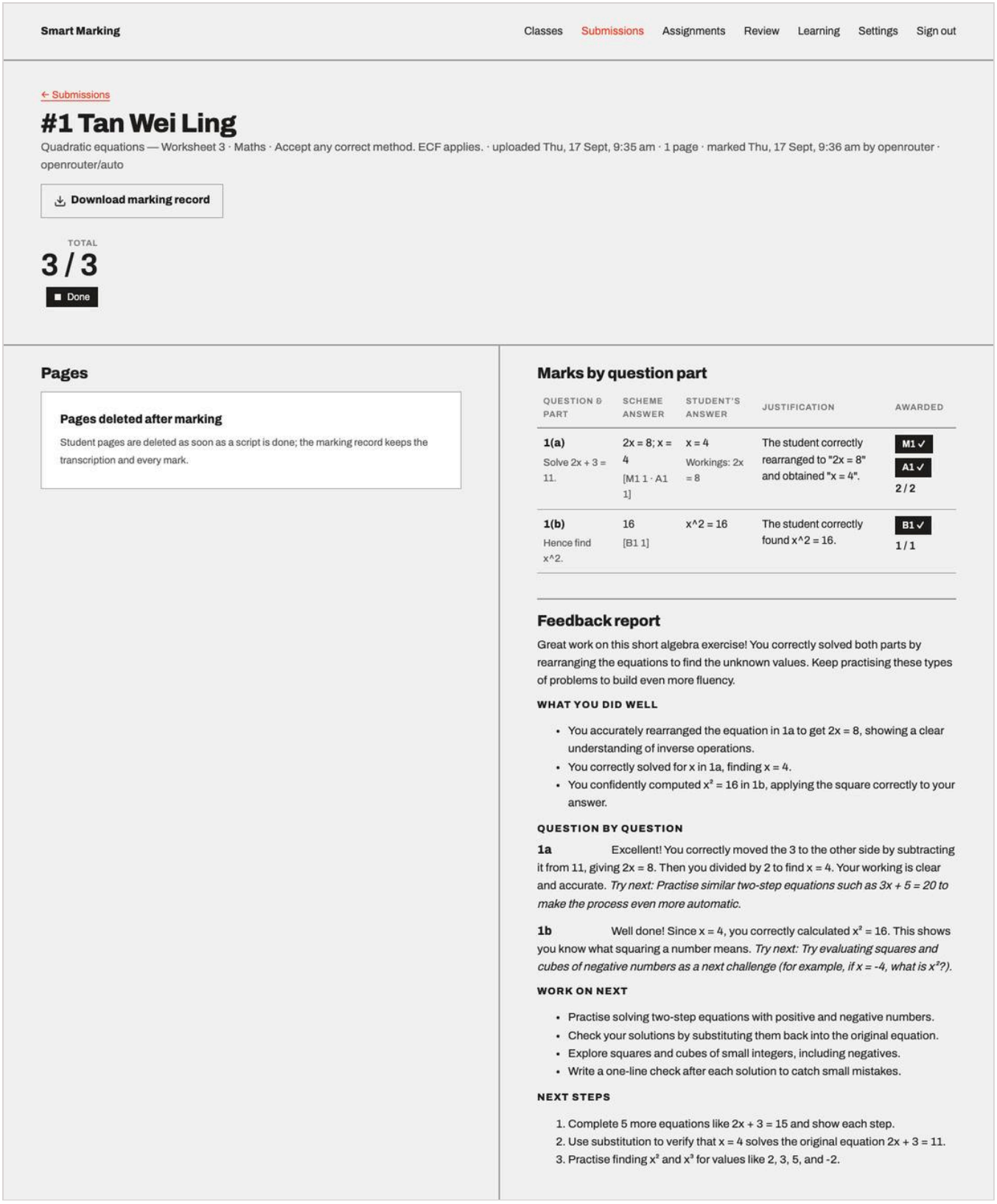
Review opens on the marking desk: the flagged tray holds the parts that need you, the ticked tray the class sets ready to release, and the drawers open Parts and Corrections.

Click a student's name to see the per-part marks, the transcription and the justification for each mark.

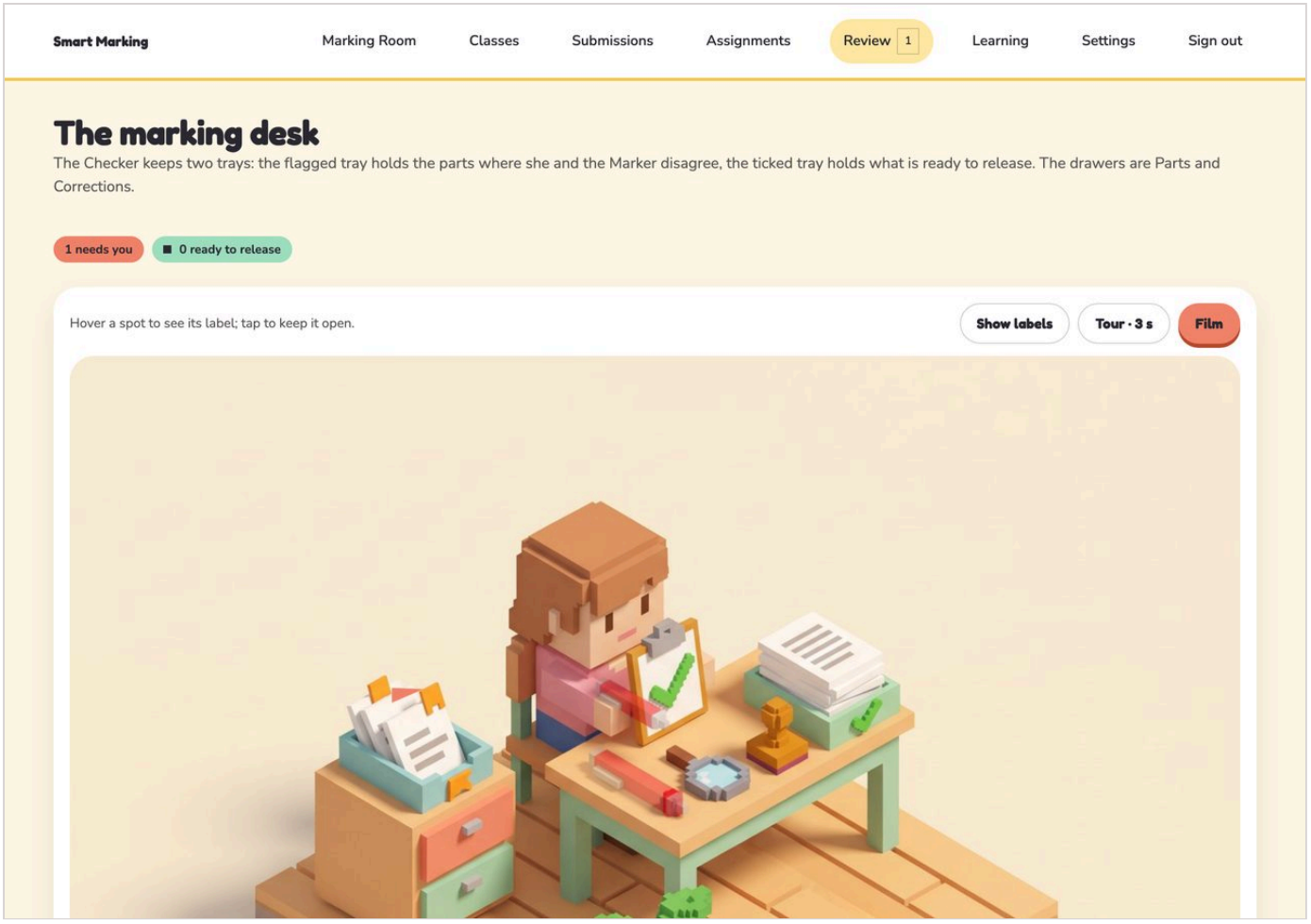
An error costs marks once. If the same slip would lose marks in two places, the reviewer flags it and the app keeps the deduction where it first happened, restoring the later one — or sends it to Review as *double penalty* when it cannot tell which allocation to restore.



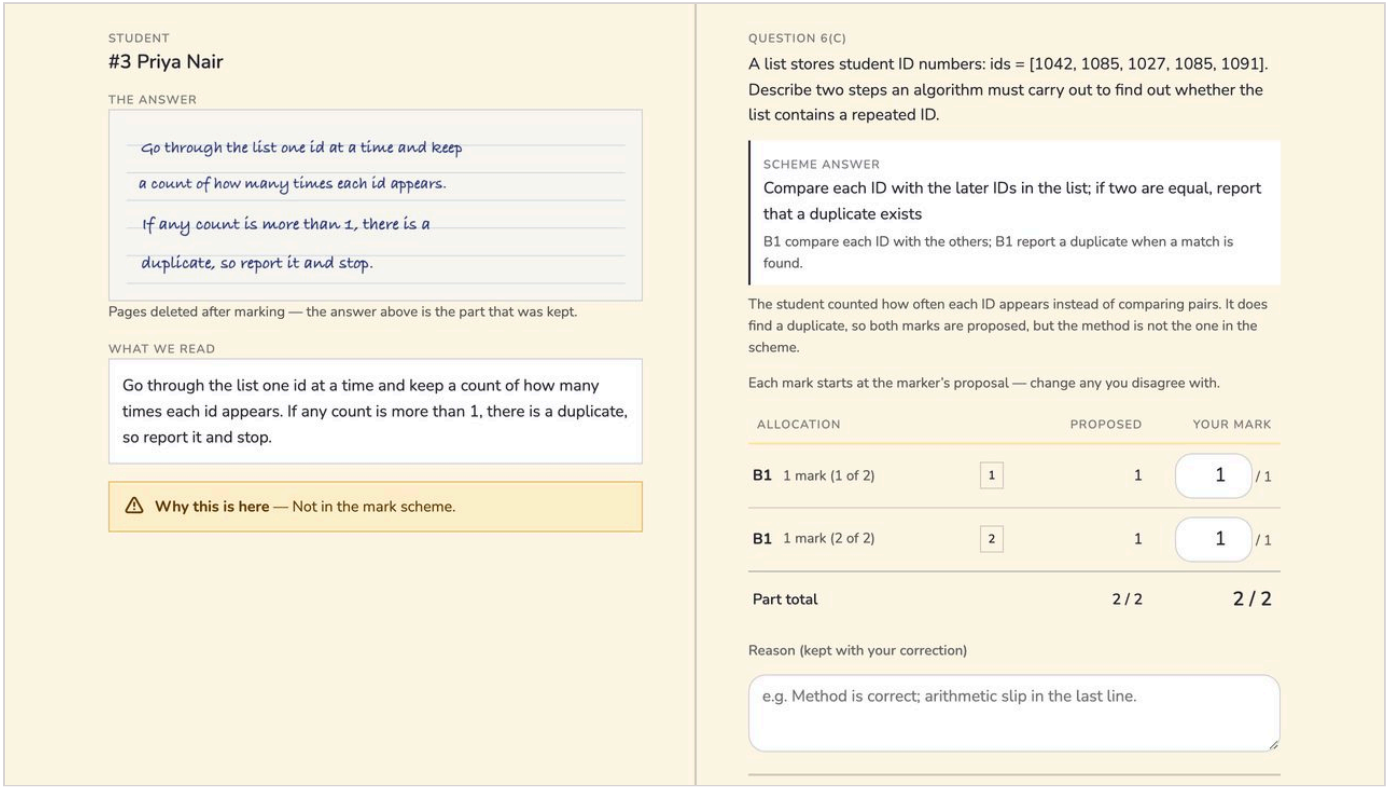
The roster after marking: 1 ready, 0 needing you.



A marked script, part by part.



Review is the marking desk: the flagged tray holds what needs you.

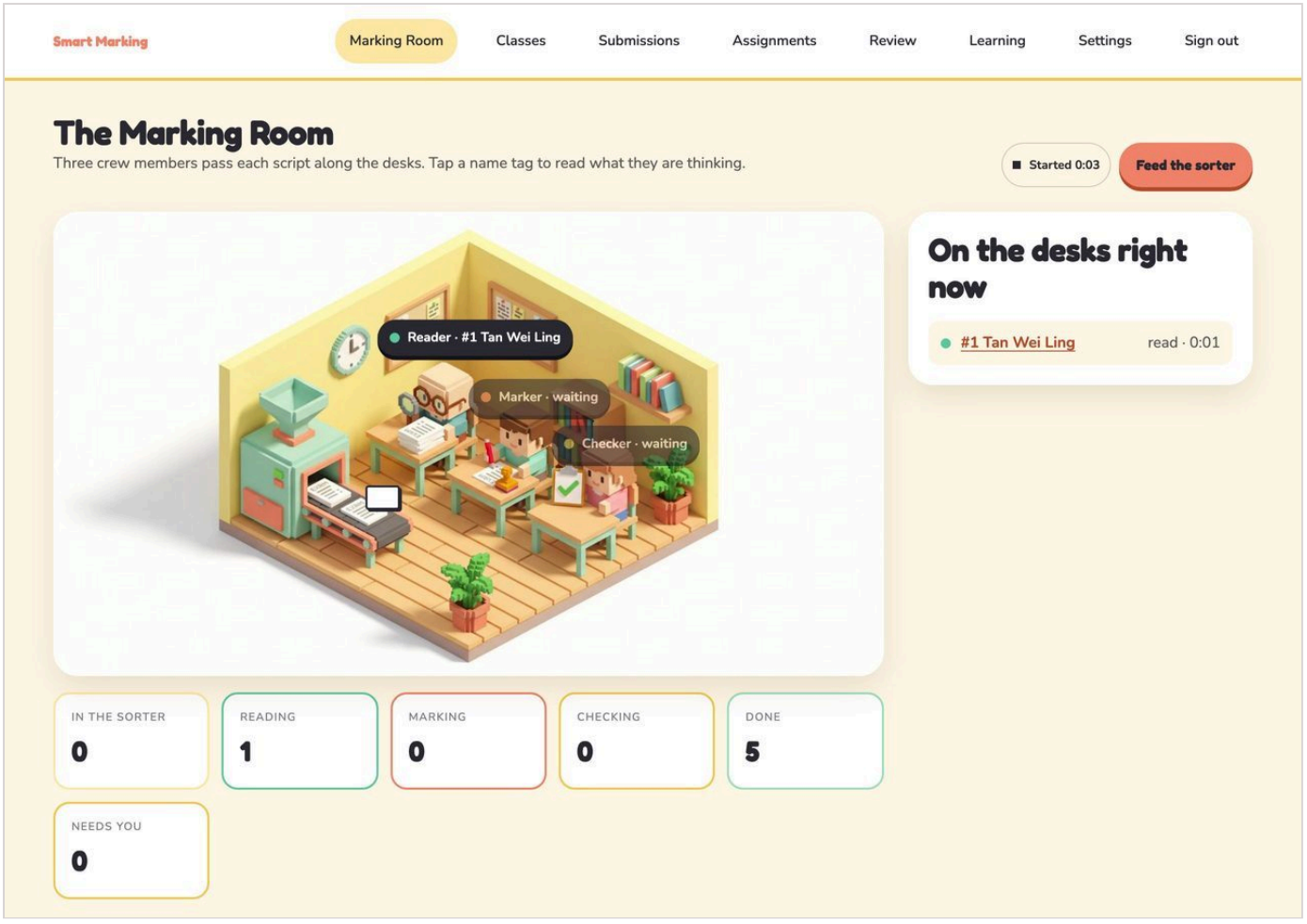


A part to decide: the student's answer, the scheme, and a mark box per allocation that starts at the proposal. This scheme gives B1 twice.

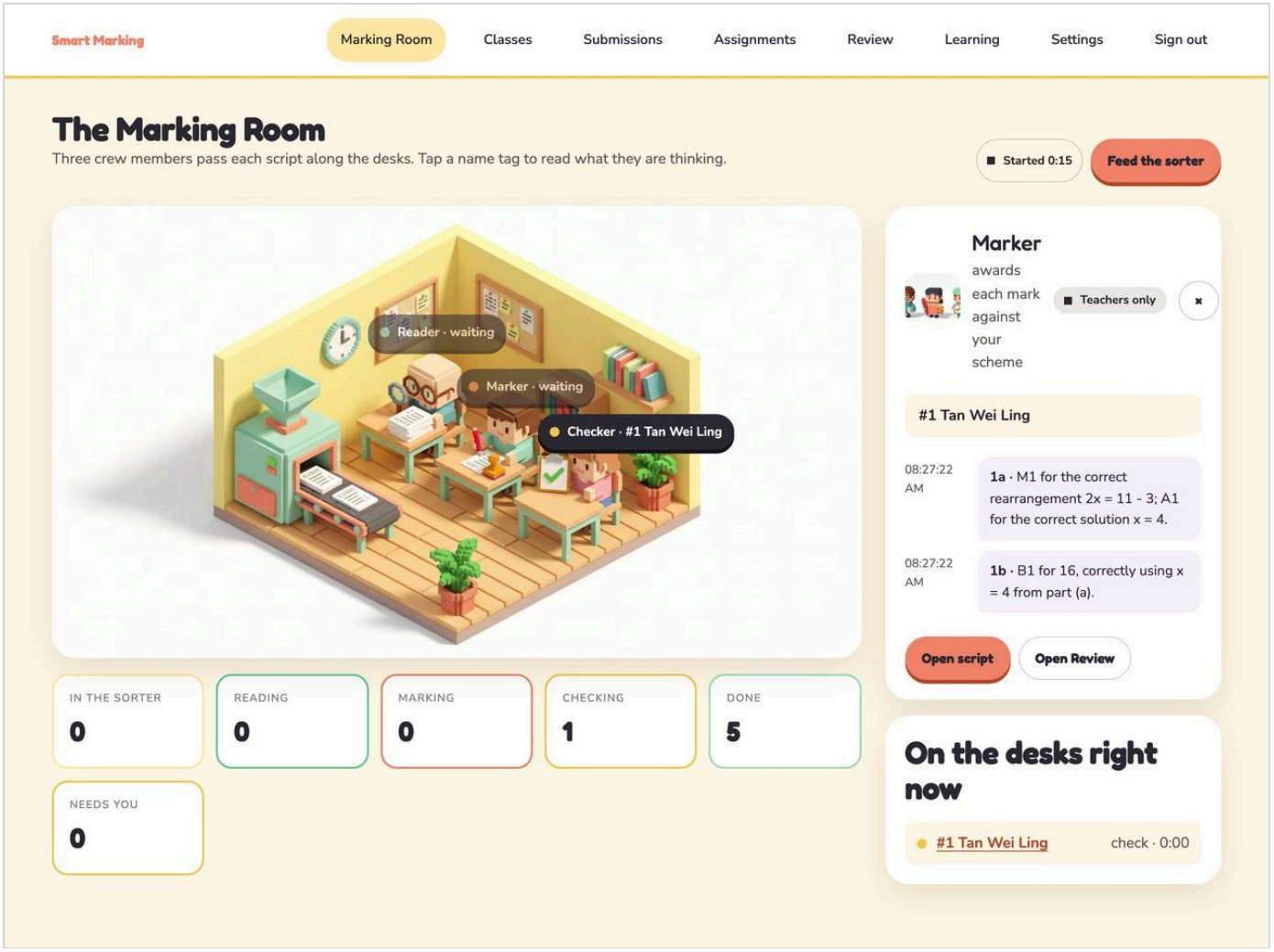
STEP 13 Watch the crew in the Marking Room

Open **Marking Room** (it is now the first page after sign-in). The sorter on the left holds scripts waiting their turn; the three desks show who is working on what: the **Reader** finds each part on the page and transcribes it, the **Marker** awards marks against your scheme, and the **Checker** re-marks independently and flags anything the two disagree on. The tiles underneath count the set as it moves through.

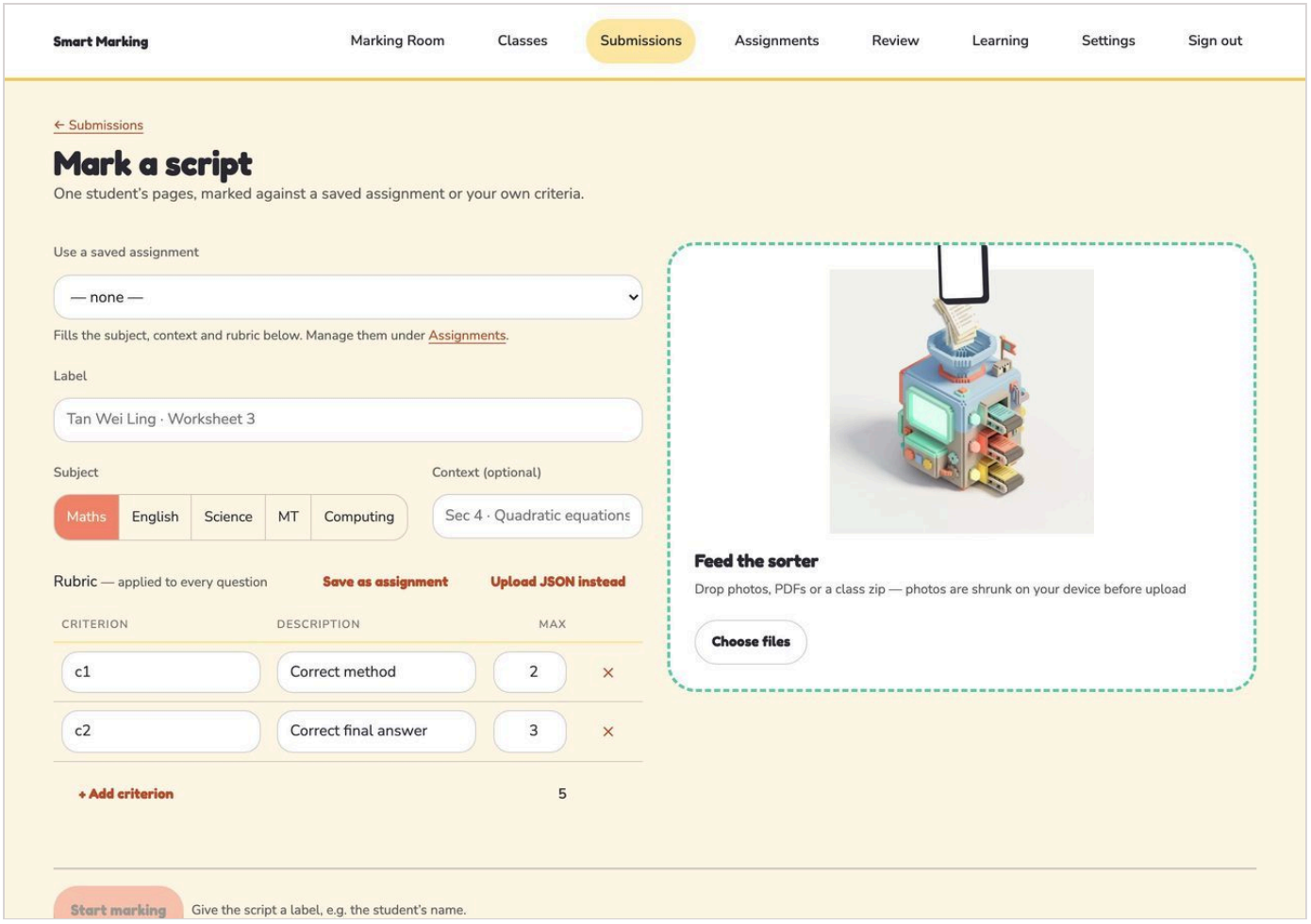
Tap a name tag to read that crew member's notes for the script on its desk, with timestamps — the Marker's justification per part, or the Checker explaining why it flagged 1(b). Nothing here is ever shown to students. The room updates live; if the feed drops it refreshes every five seconds instead, and says so.



The Marking Room while a script is on the desks — tap a tag to open the thought panel.



The Marker's notes for the script on its desk, teachers only.

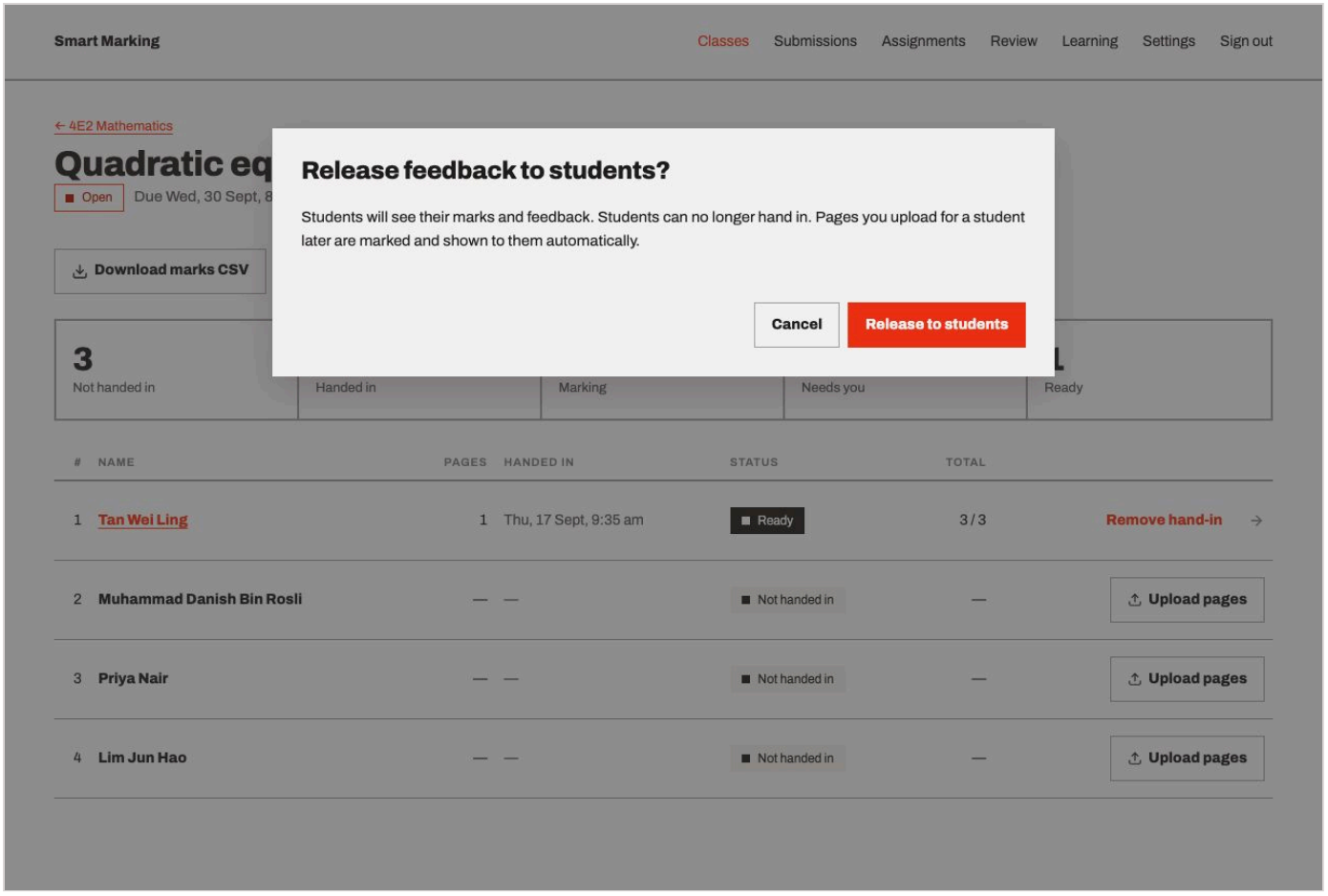


Hand in: drop photos, PDFs or a class zip into the sorter.

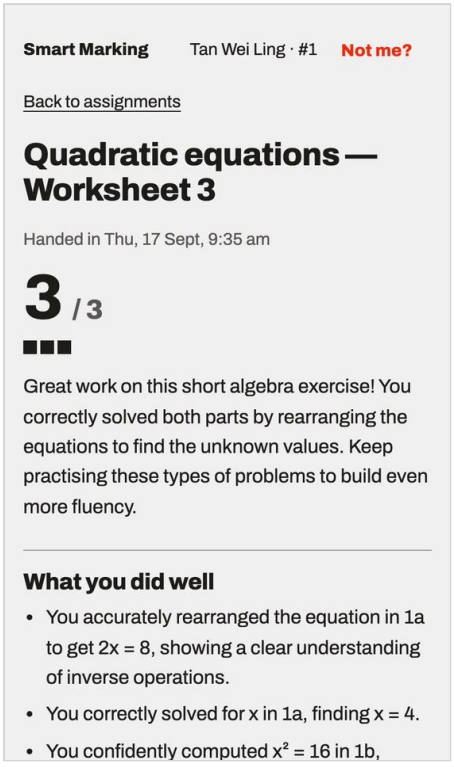
STEP 14 Release feedback

Until you release, students see "Marked — your teacher is checking". **Release feedback** is one action for the whole class and needs every "Needs you" part cleared first. After release students see their total, a short summary, what they did well, each question with a comment and a "Try next", and what to work on.

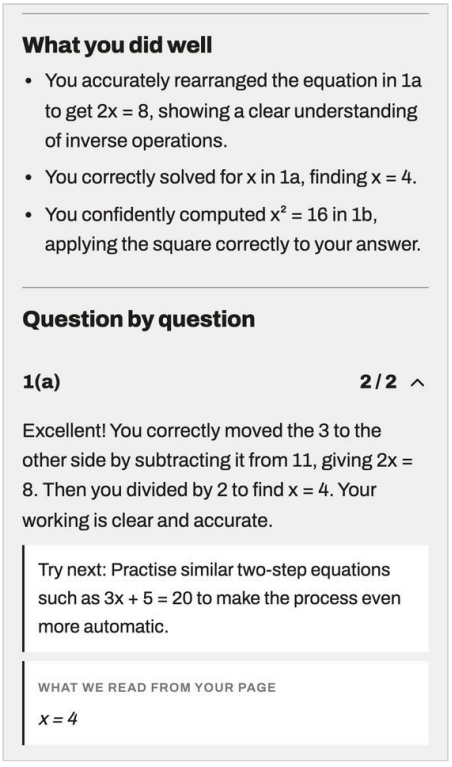
Releasing closes student hand-ins for that assignment; pages you upload for a student later are still marked and shown automatically.



The release confirmation.



What the student sees.



A question expanded.

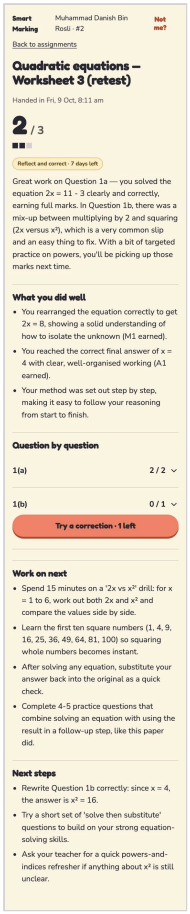
Students reflect and correct

STEP 15

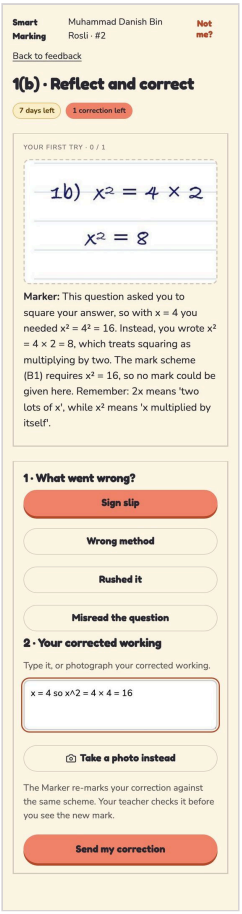
Once feedback is released, each student has a window (7 days unless you change it under **Settings → Reflection window** or on the assignment; 0 turns it off) to send **one correction per part** they lost marks on. On their phone they tap **Try a correction**, pick what went wrong, and type or photograph their corrected working.

The Marker re-marks the correction against the same scheme. You decide under **Review → Corrections**, where you see the first try, the crop of the original answer, the re-mark and the Marker's note — accept it, override the mark, or reject it — and press **Release corrections** when you are ready. The student's feedback then shows "After reflection" beside that part, and the marking record and marks CSV gain an after-reflection column.

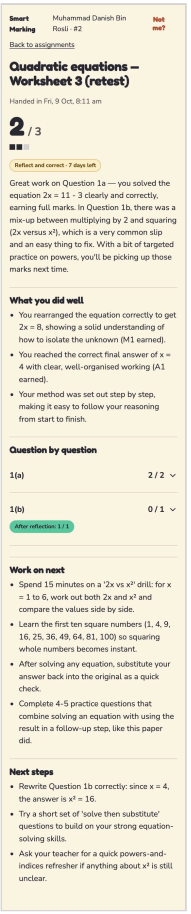
Assignments released before this version got a window of 0 days, so nothing opens unexpectedly; set a window on an assignment if you want to offer it.



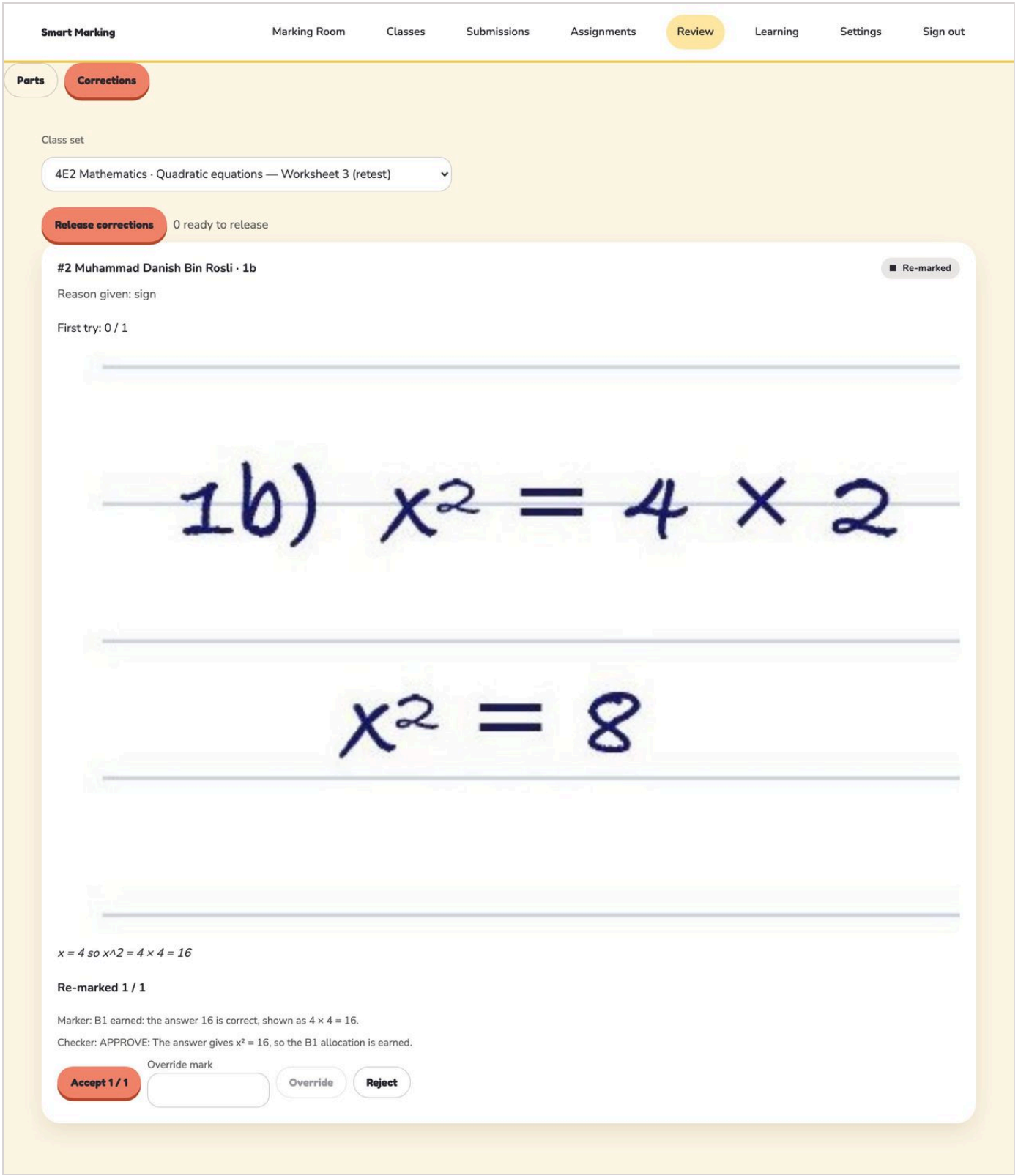
Released feedback with the reflection window and Try a correction.



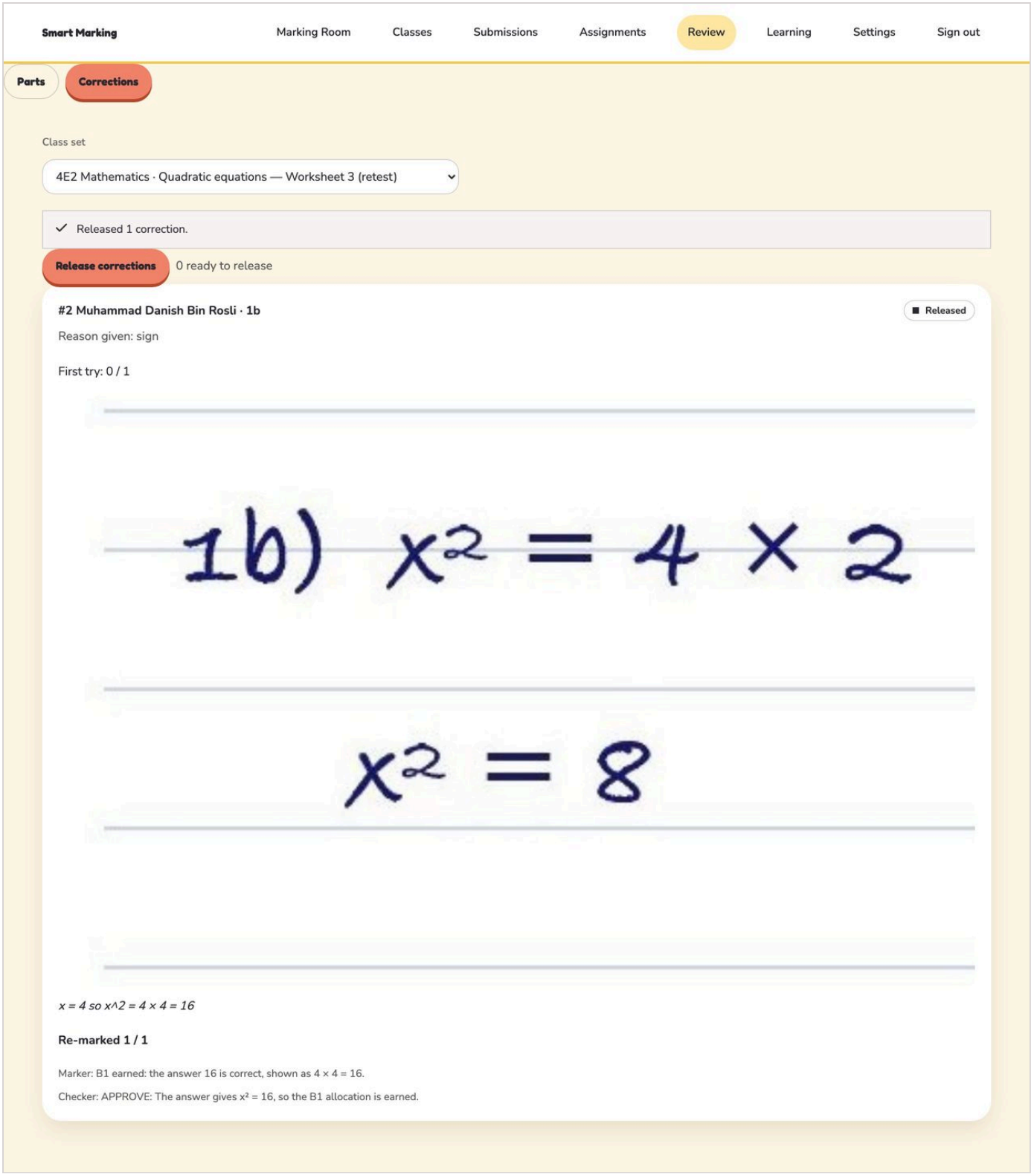
The reflect screen: first try, the Marker's comment, what went wrong, corrected working.



After release: the new mark beside the part.



Review → Corrections: first try, crop, re-mark and note, with Accept, Override and Reject.

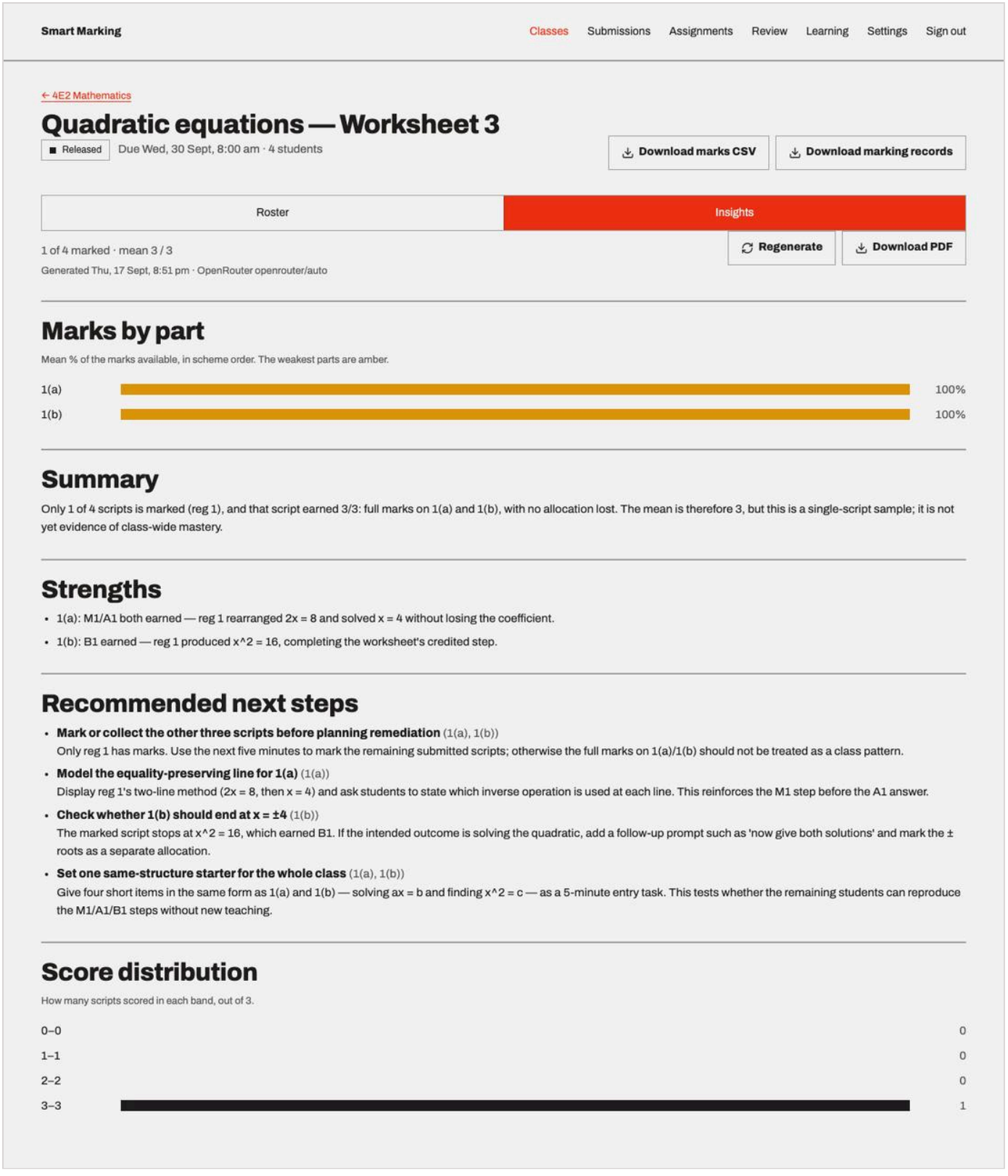


Released — the student now sees the new mark.

STEP 16 Insights: where the class lost marks

Once the first script is marked, the assignment's **Insights** tab shows the class as a whole: marks by part with the weakest highlighted, the allocations most often lost, a score distribution, and which students are struggling on which parts.

When the whole set has been marked, and again when you release, the app asks the model for a short narrative written for you, not for students — a summary, strengths, gaps and recommended next steps. Student names never reach the model: it sees register numbers and the app joins the names back. **Regenerate** refreshes the narrative after more scripts come in; **Download PDF** gives a one-file report for a department meeting.

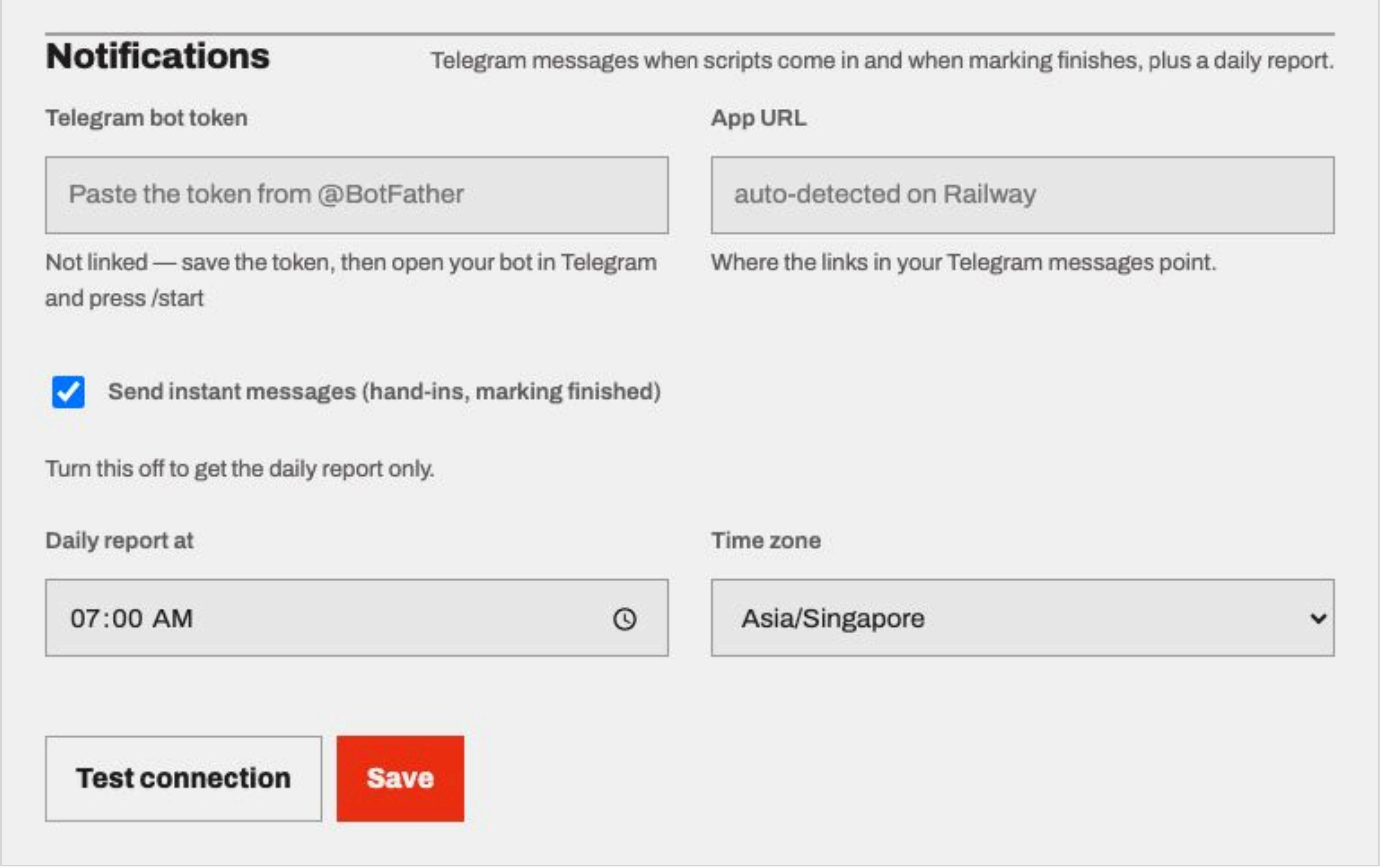


The Insights tab: numbers first, then the narrative and the students to support.

STEP 17 Telegram notifications (optional)

Get a message when scripts come in and when marking finishes, plus a daily report. In Telegram, message **@BotFather**, send `/newbot`, and copy the token it gives you. In **Settings → Notifications** paste the token and **Save**, then open your new bot in Telegram and press **Start** — within a few seconds the status changes to *Linked* ✓. **Test connection** sends a test message.

You get an instant message for new hand-ins (batched) and one when a class set finishes marking, each with links back to the app. The daily report arrives at the time you set — 07:00 Singapore time by default — and only on days something happened. Turn instant messages off to keep just the daily report. One chat per deployment: a department group works well — add the bot to the group and press `/start` there.



Settings → Notifications: token, link status, instant messages, daily report time and time zone.

STEP 18 Marks and records

From the assignment page, **Download marks CSV** gives one row per student with a column per question part, and **Download marking records** gives a Word document

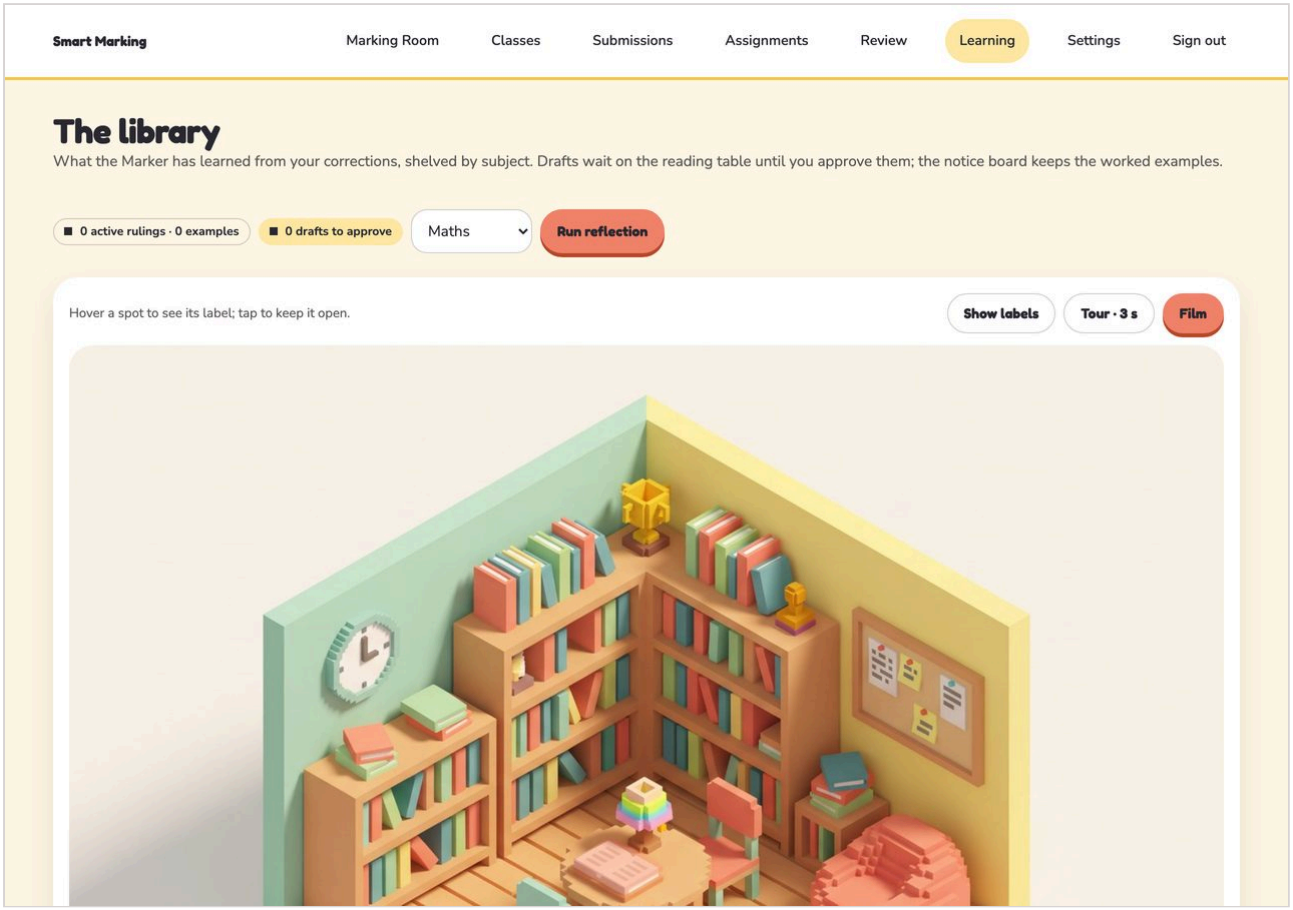
per student — mark scheme row by row, the student's answer, the awarded mark and a blank "Teacher's mark" column for moderation — plus a markbook spreadsheet.

STEP 19 Good to know

What is kept after marking. By default the app keeps the *answer portion* of each part — the region of the page the reader located it on, padded a little — and deletes the full pages once a script is done, so you can check any mark against the student's actual writing with **Show answer** on the submission page (and in Review). Choose *Keep whole pages* under Settings (or per assignment) to keep every photo, or *Delete everything* to keep only the transcription. Crops stay until you remove the hand-in.

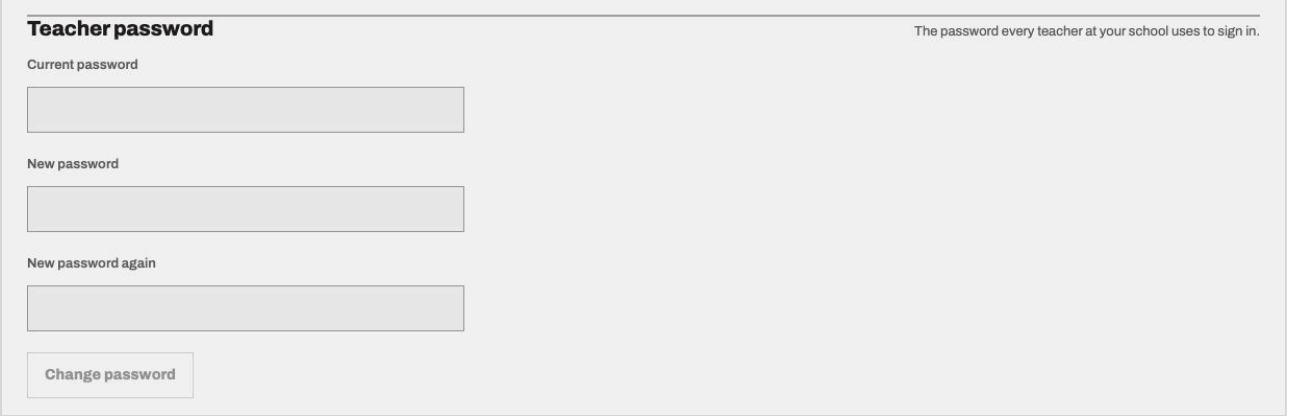
Cost. A script needs about four model calls. On a paid provider that is typically well under one cent per script for a small model; on a free tier it is free within the daily limit.

Learning is the library. What the marker has learned from your corrections is shelved by subject. Drafts wait on the reading table until you approve them, and the notice board keeps the worked examples.



Learning is the library: rulings shelved by subject, drafts on the reading table, examples on the notice board.

Changing the password or the secret. Change the teacher password under **Settings → Teacher password**. If a `TEACHER_PASSWORD` variable is set on Railway it wins over the website password (that is also the reset path). Leave `SECRET_KEY` alone — it encrypts the stored API keys; changing it means re-entering them.



Settings → Teacher password.

File limits. 12 files per submission, 2 MB per program file, 20 MB per zip and 50 MB per upload; `.xlsm` and macros are refused, and nothing you upload is ever executed.

Telegram and Insights are optional. Nothing is sent to Telegram until you link a bot, and Insights only use the model you already configured — no extra keys.

Updates. Redeploying the `web` service picks up the latest version from GitHub; the database and stored keys are kept.

Source, issues and the full README: github.com/twahidin/smart-marking-system.

Smart Marking is open source: github.com/twahidin/smart-marking-system · Deploy: railway.com/deploy/smart-marking-1